

Tokenizační řešení pro Multikanálový odbavovací systém

Historie změn	4
Obsah a cíl dokumentu	6
Seznam zkratk	6
Výklad pojmů	7
Multikanálový odbavovací systém	7
Operátor	7
Dopravce	7
Identifikátor	7
Typ identifikátoru	7
Tokenizační autorita	8
Token	8
Tokenizační algoritmus	8
Tokenizační klíč	8
Tokenizace	8
Tokenizační procesor	8
Tokenizační brána	8
Acquirer	8
MOS Frontend	9
MOS Backend	9
Kontaktní místo MOS	9
Registrační terminál	9
Prodejní terminál	9
Obslužná aplikace (Pokladna)	9
Verifikace identifikátoru	9
Odbavovací zařízení	9
Základní členění navrhovaného systému	10
Byznys procesy	10
Registrace identifikátoru přes tokenizační bránu	10
Registrace identifikátoru přes registrační terminál	11
Jednotlivá tokenizace přes API tokenizačního procesoru	12
Dávková tokenizace	12
Opětovná verifikace existujícího identifikátoru	13

Správa tokenu a správa identifikátoru	13
Zavedení nového typu identifikátoru	13
Zavedení nové sady tokenizačních parametrů	14
Zneplatnění tokenizačního klíče	15
Tokenizace	15
Tokenizační algoritmus	15
Tokenizační algoritmus - verze 1	15
Tokenizační algoritmus - verze 16	17
Platnost tokenizačních klíčů	19
API pro tokenizaci	20
Operace	20
Návratové kódy	20
Zabezpečení pomocí signature	20
Datové typy	21
Návrh API pro registrační požadavek pro tokenizační bránu	22
POST /api/v1/registration	22
Požadavek (povinné parametry jsou zvýrazněné)	22
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	22
Příklad volání (testovací prostředí)	23
Příklad odpovědi (úspěšně založený registrační požadavek)	23
Příklad odpovědi (nevalidní vstupní parametry)	23
GET /api/v1/registration/{regId}	24
Požadavek (povinné parametry jsou zvýrazněné)	24
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	24
Příklad volání (testovací prostředí)	25
Příklad odpovědi	25
Návrh API pro jednotlivou tokenizaci	26
POST /api/v1/identifier	26
Požadavek (povinné parametry jsou zvýrazněné)	26
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	27
Příklad volání pro Lítačku (testovací prostředí)	28
Příklad volání pro InKartu (testovací prostředí)	28
Příklad volání pro Mobilní aplikaci (testovací prostředí)	29
Příklad odpovědi (úspěšně provedená tokenizace nového identifikátoru)	29
Příklad odpovědi (existující identifikátor, tokenizace proběhla dříve)	30
Příklad odpovědi (nevalidní vstupní parametry)	30
Návrh API pro dávkovou tokenizaci	30
Návrh formátu souborů pro dávkovou tokenizaci	31

Návrh formátu souborů obsahujícího blacklist Lítačky	33
Návrh formátu souborů obsahujícího blacklist InKarty	34
Návrh formátu souboru obsahujícího blacklist tokenů	34
Návrh formátu mapovacího souboru pro Lítačku	35
Mapování pro InKartu	35
Návrh API pro opětovnou verifikaci existujícího identifikátoru	35
POST /api/v1/identifier/{token}/verify	35
Požadavek (povinné parametry jsou zvýrazněné)	36
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	36
Příklad volání (testovací prostředí)	36
Příklad odpovědi (identifikátor je platný)	36
Příklad odpovědi (identifikátor není platný, obecné zamítnutí)	37
Návrh API pro správu tokenů a identifikátorů	37
GET /api/v1/token/{token}	37
Požadavek (povinné parametry jsou zvýrazněné)	37
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	37
Příklad volání (testovací prostředí)	38
Příklad odpovědi	38
PUT /api/v1/token/{token}	38
Požadavek (povinné parametry jsou zvýrazněné)	38
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	39
Příklad volání (testovací prostředí)	39
Příklad odpovědi (úspěšně provedená operace)	39
Příklad odpovědi (nepovolená hodnota vstupního parametru)	40
Příklad odpovědi (nelze provést operaci, token není ve správném stavu)	40
DELETE /api/v1/token/{token}	40
Vstupní parametry (povinné parametry jsou zvýrazněné)	40
Příklad volání (testovací prostředí)	40
Příklad odpovědi	41
GET /api/v1/identifier/{token}?mode={mode}	41
Požadavek (povinné parametry jsou zvýrazněné)	41
Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)	41
Příklad volání (testovací prostředí)	43
Příklad odpovědi (defaultní mód activeTokenValues)	43
Příklad odpovědi (požadovaný mód activeTokensDetail)	43
Příklad odpovědi (požadovaný mód allTokensDetail)	44
Příklad odpovědi (nepovolená hodnota vstupního parametru)	44
PUT /api/v1/identifier/{token}	45
Vstupní parametry (povinné parametry jsou zvýrazněné)	45

Návratové hodnoty (parametry, které budou vráceny vždy, jsou zvýrazněné)	45
Příklad volání (testovací prostředí)	46
Příklad odpovědi (úspěšně provedená operace)	46
Příklad odpovědi (nepovolená hodnota vstupního parametru)	46
Příklad odpovědi (nelze provést operaci, identifikátor není ve správném stavu)	46
DELETE /api/v1/identifier/{token}	47
Vstupní parametry (povinné parametry jsou zvýrazněné)	47
Příklad volání (testovací prostředí)	47
Příklad odpovědi	47
Číselníky	47
Typ identifikátoru	47
Typ Lítačky	48
Typ BPK	48
Chybový kód	48
Výsledek verifikace existujícího identifikátoru	49
Životní cyklus	49
Registrační požadavek	49
Identifikátor	51
Token	52
Tokenizační klíč	53
Tokenizační brána	54
Validace Lítačky/OpenCard	56
Registrační terminál	57
Architektura systému	58
Logická architektura	58
Fyzická architektura	59
Napojení na okolní systémy	61
Transfer dat do backendu MOS	61
Verifikace identifikátoru pro jednotlivé dopravce	64

Historie změn

Verze	Autor	Popis
1.0	D. Marek	Iniciální verze
1.1	D. Marek	Doplněn nový byznys proces a API operace pro „opětovnou verifikaci pro existující identifikátor“ a možnost odregistrace identifikátoru (nový stav v životním cyklu Identifikátoru)
1.2	D. Marek	Sjednocen formát expirace na MMYU u BPK (kap.8.1.2), doplněn popis a příklady tokenizace do kapitol 8.1.1 a 8.1.2), doplněn odkaz na TokenTransferWSv1.wsdl v kapitole 13.1, doplněn číselník v kapitole 9.7.5, upravena kapitola 9.5 (návrátový kód u opětovné verifikace existujícího identifikátoru), doplněna kapitola 9.4.3 (návrh formátu blacklist token souborů)
1.3	D. Marek	Doplněn nový typ identifikátoru - InKarta (změny zaznačeny barevně) upravena platnost tokenizačních klíčů (verze 1, 2 a 16)
1.4	D. Marek	Doplněny parametry pro registraci Lítačky u jednotlivé tokenizace v kapitole Návrh API pro jednotlivou tokenizaci
1.5	D.Marek	- Opraven překlep STP-SIGN na STP-SIGNATURE v kapitole "Zabezpečení pomocí signature" (v příkladech bylo správně uvedeno STP-SIGNATURE) - Doplněn popis stavů tokenu, které lze smazat v kapitole DELETE /api/v1/token/{token} - Upraven formát zadávání platnosti InKarty na Tokenizační bráně v kapitole Tokenizační brána (změnový požadavek) - Doplněn popis nové validace Lítačky/OpenCard v kapitole Validace Lítačky/OpenCard (změnový požadavek) - Upraven popis dávkového zpracování pro InKartu (změna platnosti na dd.mm.yyyy) v kapitole Návrh formátu souborů pro dávkovou tokenizaci - Upraven popis plnění parametru IdentRef pro Inkartu (předávání dd.mm.yyyy) u jednotlivé tokenizace v kapitole Návrh API pro jednotlivou tokenizaci - Doplněn popis byznys procesu dávkové tokenizace v kapitole Dávková tokenizace a detailní specifikace v kapitole Návrh formátu souborů pro dávkovou tokenizaci - změna předávání platnosti InKarty do backendu MOS (dd.mm.yyyy) v kapitole Transfer dat do backendu MOS
1.6	D.Marek	Doplněny parametry maskCln a cardExp do GET /api/v1/registration/...

1.7	D.Marek	Doplněn nový typ identifikátoru Mobilní aplikace (MA) - doplněn seznam zkratk - doplněný popis tokenizace pro algoritmus v1 a v16 (včetně příkladů pro mobilní aplikaci) - doplněn příklad u jednotlivé tokenizace - doplněn číselník typů identifikátorů (mobapp) - doplněn popis pro tokenizační bránu (zaintegrovaní návodu pro použití mobilní aplikace) - doplněn popis integrace na backend MOS (posílání tokenizovaných dat - typ identifikátoru mobilní aplikace).
1.8	J. Gajdošík	Doplněn typ identifikátoru Opus Card

Obsah a cíl dokumentu

Tento dokument obsahuje návrh tokenizačního řešení pro Multikanálový odbavovací systém.

Popisuje následující okruhy:

- Základní charakteristika navrhovaného systému, definice pojmů
- Základní členění tokenizačního řešení
- Popis byznys procesů vztahujících se k tokenizačnímu řešení pro MOS
- Návrh tokenizačních algoritmů, popis parametrů pro tokenizaci
- Detailní návrh API
- Tokenizační brána
- Registrační terminál
- Architektura systému
- Napojení na okolní systémy

Cílem je vyspecifikovat chování a rozhraní navrhovaného systému s ohledem na jeho budoucí rozšiřování.

Seznam zkratk

MOS	Multikanálový odbavovací systém
DPP	Dopravní podnik Hlavního města Prahy
BPK	Bezkontaktní platební karta vydaná MasterCard či Visa

BČK	Bezkontaktní čipová karty používaná v dopravě, nejčastěji Mifare technologie
MC	MasterCard
PCI-DSS	Payment Card Industry Data Security Standard
3DS	Standard asociací Visa a MasterCard, vystupující pod komerčními názvy MasterCard SecureCode a Verified by Visa. Zajišťuje vyšší bezpečnost plateb bankovními kartami.
SAM	Secure access module
UID	Hardware číslo BČK přiřazené výrobcem karty (výrobní číslo čipu)
CLN	Logické číslo BČK přiřazené vydavatelem karty a viditelné pro držitele
PAN	Primary account number, číslo bankovní platební karty
API	Application Interface (programové rozhraní)
DB	Databáze
HSM	Hardware security module
ODA	Offline data authentication, offline způsob ověření validity bankovní platební karty
STP	Smart Token Processor
MA	Mobilní aplikace

Výklad pojmů

Multikanálový odbavovací systém

Nově budovaný systém pro odbavování cestujících ve veřejné dopravě.

Operátor

Operátor ICT, a. s., zadavatel systému Multikanálového odbavovacího systému, koordinuje napojení dopravců do systému MOS.

Dopravce

Subjekt zajišťující dopravu cestujících ve veřejné dopravě, dopravce integrovaný do systému MOS bude např. DPP.

Identifikátor

Obecný pojem pro kartu, přívěšek či jiný předmět sloužící cestujícímu pro odbavení ve veřejné dopravě.

Typ identifikátoru

Typ identifikátoru z hlediska byznys aplikace: *Lítačka*, *BPK*, *InKarta*, *Mobilní aplikace*, *OpusCard*

Tokenizační autorita

Stanovuje obecné a bezpečnostní parametry pro tokenizaci v souladu s ohledem na PCI-DSS a platnou legislativu. Tuto roli bude vykonávat Operátor ICT, a. s.

Token

Zástupná hodnota identifikátoru. Zatímco hodnota identifikátoru (např. PAN pro BPK) může být citlivým platebním údajem, token není citlivou informací a jeho vyzrazení nemůže způsobit finanční či jinou újmu.

Tokenizační algoritmus

Specifikace / postup výpočtu tokenu ze vstupních dat: hodnoty identifikátoru a z tokenizačního klíče.

Tokenizační klíč

Tajemství, které vstupuje do tokenizace. Souhrnný pojem pro klíče symetrických algoritmů, privátních či veřejných klíčů nesymetrické kryptografie nebo salt (sůl) pro hashovací funkce.

Tokenizace

Proces, během kterého se z citlivého platebního údaje vypočte token.

Tokenizační procesor

Systém, který běží v PCI DSS prostředí, je napojený na HSM, zabezpečeným způsobem uchovává identifikátory a provádí vlastní tokenizaci podle pravidel tokenizační autority.

Tokenizační brána

Systém pro zákazníky obsahující webové rozhraní umožňující bezpečným způsobem zadat citlivou hodnotu identifikátoru pro provedení registrace. Tokenizační brána je napojena na tokenizační procesor, který provádí vlastní tokenizaci.

Acquirer

Subjekt, který poskytuje online službu pro ověření bankovních platebních karet.

MOS Frontend

Systém pro zákazníky DPP obsahující webové rozhraní umožňující nákup jízdného.

MOS Backend

Centrální komponenta Multikanálového odbavovacího systému pro evidenci zákazníků a jejich jízdného poskytující služby pro dopravce, např. whitelisty cestujících s platným jízdním dokladem.

Kontaktní místo MOS

Obecné označení pro kontaktní místo pro zákazníky jednotlivých dopravců napojených na systém MOS.

Registrační terminál

Terminál, který je propojený s tokenizačním procesorem umožňující registraci nového identifikátoru, tento terminál neprovádí žádné platební operace.

Prodejní terminál

Terminál, který je propojený na systém dopravce, na terminálu je možné provést platbu.

Obslužná aplikace (Pokladna)

Aplikace umístěná na kontaktním místě MOS, pomocí které obsluha realizuje nákup jízdného. Obslužná aplikace je propojena s prodejním terminálem, není propojena s registračním terminálem.

Verifikace identifikátoru

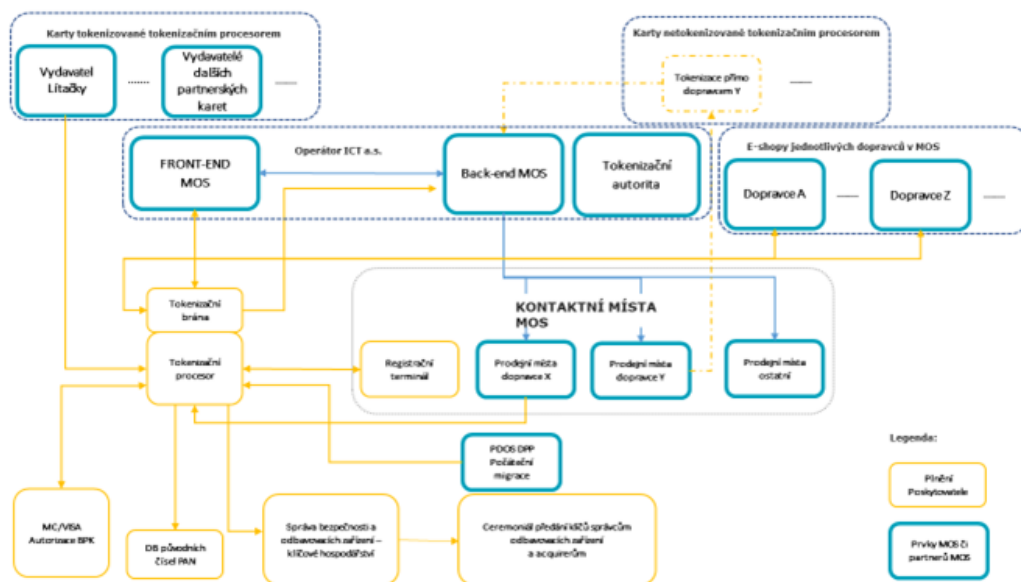
Ověření validity identifikátoru u dopravce, který identifikátor vydal.

Odbavovací zařízení

Odbavovací zařízení, umístěné zpravidla ve vozidle hromadné dopravy, provádí tokenizaci přiloženého identifikátoru a ověření validity (např. oproti whitelistu).

Základní členění navrhovaného systému

Navrhovaný systém tokenizačního řešení se skládá z *tokenizačního procesoru*, *tokenizační brány* a *registračního terminálu*.



Navrhovaný systém bude posílat data do backendu MOS, bude napojen na systémy dopravců. K API navrhovaného systému budou přistupovat dopravci a operátor, na tokenizační bránu budou napojeny eshopy dopravců a Frontend MOS.

Tokenizační procesor bude bezpečným způsobem v databázi uchovávat citlivá data: hodnoty identifikátorů, bude bezpečným způsobem v HSM uchovávat tokenizační klíče a bude provádět tokenizaci identifikátorů.

Byznys procesy

Registrace identifikátoru přes tokenizační bránu

Systém umožňuje integraci eshopu dopravce (např. Frontendu MOS) na tokenizační bránu za účelem registrace identifikátoru.

Zákazník na eshopu dopravce zvolí jízdné, které chce zakoupit (např. roční kupón), eshop dopravce jej poté přesměruje na tokenizační bránu, kde zákazník zadá identifikátor, který bude používat ve veřejné dopravě. Tokenizační brána ověří platnost zadaného identifikátoru a provede pomocí tokenizačního procesoru vlastní tokenizaci. Vypočtené tokeny předává tokenizační procesor do backendu MOS. Zákazník je z tokenizační brány následně přesměrován zpět na eshop dopravce a proces nákupu jízdného je dokončen.

Registrace identifikátoru přes tokenizační bránu se týká jak BPK, tak non-BPK identifikátorů. V případě BPK se provádí ověření platební karty pomocí acquirera (např. 3DS ověření u vydavatele karty), pro non-BPK identifikátory systém umožňuje provést verifikaci identifikátoru u dopravce (ověří se například, že daný identifikátor existuje, je platný a není na blacklistu apod).

Ve výsledku má zákazník zakoupené jízdné, které je spárované s jeho zadaným identifikátorem. Identifikátor zákazník používá pro odbavení ve veřejné dopravě a prokazuje se jím při revizorské kontrole.

Detailní technická specifikace je uvedena v kapitole [Návrh API pro registrační požadavek pro tokenizační bránu](#), pro registraci identifikátoru přes tokenizační bránu použije dopravce následující API operace:

POST /api/v1/registration	(založení registračního požadavku)
GET /api/v1/registration/{regId}	(vyčtení výsledku registračního požadavku)

Vlastní přesměrování z eshopu dopravce na tokenizační bránu a zpět je provedeno pomocí redirectu v prohlížeči zákazníka.

Registrace identifikátoru přes registrační terminál

Systém umožňuje registraci identifikátoru na kontaktním místě MOS na registračním terminálu.

V případě, že token pro daný identifikátor neexistuje v backendu MOS, provede zákazník přiložení identifikátoru k registračnímu terminálu, ten provede vyčtení hodnoty identifikátoru, přičemž systém provede ověření a následnou tokenizaci daného identifikátoru. Vypočtené tokeny předává tokenizační procesor do backendu MOS. Zákazník na kontaktním místě dokončí nákup jízdného (provede přiložení identifikátoru k prodejnímu terminálu, kde je vypočten token), provede se spárování tokenu (jeho identifikátoru) se zakoupeným jízdným v backendu MOS.

Registrace identifikátoru přes registrační terminál se týká jak BPK, tak non-BPK identifikátorů. V případě BPK se provádí ověření platební karty pomocí ODA, pro non-BPK identifikátory systém umožňuje provést verifikaci (ověří se například, že daný identifikátor existuje, je platný a není na blacklistu apod).

Registrační terminál bude umístěn na kontaktním místě MOS, vlastní aktivace čtečky terminálu bude aktivována přes hotkey na klávesnici terminálu. Registrační terminál nebude propojen s pokladnou (obslužnou aplikací).

Jednotlivá tokenizace přes API tokenizačního procesoru

Tokenizační procesor obsahuje API pro provádění tzv. *jednotlivé tokenizace*. Přístup k API má dopravce, který pomocí tohoto API provádí tokenizaci nově vydávaných non-BPK identifikátorů.

Detailní technická specifikace je uvedena v kapitole [Návrh API pro jednotlivou tokenizaci](#), pro registraci identifikátoru přes tokenizační bránu použije dopravce následující API operace:

`POST /api/v1/identifier` (provede uložení a tokenizaci identifikátoru)

Vypočtené tokeny předává tokenizační procesor do backendu MOS. Hodnoty jsou předávány asynchronně, nečeká se na dokončení předání dat do backendu MOS, v případě nedostupnosti backendu MOS se data předají po obnovení spojení.

Dávková tokenizace

Tokenizační procesor obsahuje rozhraní pro tzv. *dávkovou tokenizaci*. Přístup k rozhraní má dopravce, který provádí iniciální a případný následný import existujících identifikátorů, které je potřeba tokenizovat.

Dávková tokenizace je primárně určena pro velké objemy dat, kdy není vhodné jednotlivě provolávat API tokenizačního procesoru a provádět jednotlivou tokenizaci. Přes toto rozhraní se dále předávají soubory s blacklisty a s mapováním specifické pro Lítačku.

V dávkové tokenizaci předává dopravce identifikátory určené k tokenizaci v tzv. *vstupním souboru* splňujícím předem dohodnutý XML formát. Tokenizační procesor provede zpracování identifikátorů, jejich tokenizaci, následné předání tokenů do backendu MOS a vytvoří tzv. *výstupní soubor* obsahující vygenerované tokeny splňující předem dohodnutý XML formát. Proces končí předáním výstupního souboru dopravci.

Vypočtené tokeny předává tokenizační procesor do backendu MOS, při generování výstupního souboru se nečeká na předání všech hodnot do backendu MOS (předání probíhá asynchronně, postupně, v menších objemech). V případě nedostupnosti backendu MOS se data předají po obnovení spojení.

V případě dávkové tokenizace Lítačky nebude prováděna synchronizace tokenizačních dat do backendu MOS přes online rozhraní, data se budou předávat pouze pomocí výstupního souboru doplněného o potřebné parametry *platnost od*, *platnost do* a *typ lítačky* (změnový požadavek).

Detailní technická specifikace je uvedena v kapitole [Návrh API pro dávkovou tokenizaci](#).

Opětovná verifikace existujícího identifikátoru

API tokenizačního procesoru obsahuje operaci, pomocí které může dopravce anebo operátor provádět opětovnou verifikaci existujícího identifikátoru. Ověření se provede například v situaci, kdy zákazník si na existující identifikátor, na který si dříve koupil roční kupón chce nově koupit další například roční kupón.

Verifikace bude probíhat podle pravidel pro daný identifikátor (pro non-BPK ověření proti blacklistu u daného dopravce, pro BPK ověření karty u acquirera).

Detailní technická specifikace je uvedena v kapitole [Návrh API pro opětovnou verifikaci existujícího identifikátoru](#), dopravce použije následující API operaci:

POST /api/v1/identifier/{token}/verify (provede verifikaci identifikátoru)

Správa tokenů a správa identifikátorů

API tokenizačního procesoru obsahuje operace, pomocí kterých může dopravce anebo operátor provádět správu tokenů a správu identifikátorů.

Detailní technická specifikace je uvedena v kapitole [Správa tokenů a správa identifikátorů](#), pro správu tokenů a identifikátorů použije dopravce následující API operace:

GET /api/v1/identifier/{token} (získání údajů pro daný identifikátor)

PUT /api/v1/identifier/{token}	(změna stavu příslušného identifikátoru)
DELETE /api/v1/identifier/{token}	(smazání identifikátoru)
GET /api/v1/token/{token}	(získání údajů pro daný token)
PUT /api/v1/token/{token}	(změna stavu příslušného tokenu)
DELETE /api/v1/token/{token}	(smazání tokenu)

Zavedení nového typu identifikátoru

Tento proces iniciuje *tokenizační autorita*:

1. předem dohodnutým komunikačním kanálem (tel./email) předá provozovateli tokenizačního procesora potřebné údaje pro zavedení a implementaci nového typu identifikátoru:
 - Určí, zda se má hodnota nového typu identifikátoru tokenizovat
 - Dodá specifikaci jakým způsobem proběhne vyčtení a ověření hodnoty identifikátoru na terminálu nebo odbavovacím zařízení (včetně specifikace jakým způsobem pracovat se SAM, jakým způsobem provádět ověření)
 - Dodá potřebné komponenty pro vývoj a otestování nového typu identifikátoru (testovací SAM, testovací karta)
 - Dodá specifikaci, jakým způsobem se hodnota identifikátoru pro tokenizaci má naformátovat pro provedení vlastní tokenizace (tzv. *inputdata* vstupující do tokenizace, viz dále)
 - Dodá specifikaci a popis rozhraní, jakým způsobem provádět verifikaci hodnoty identifikátoru oproti backend systému dopravy
2. Dodavatel tokenizačního řešení rozšíří funkcionalitu o zadávání
 - a. nového typu identifikátoru na tokenizační bránu
 - b. upraví funkcionalitu registračním terminálu o nový způsob vyčtení nového typu identifikátoru
 - c. provede úpravu tokenizačního procesoru (nový typ identifikátoru, verifikace identifikátoru u dopravy).

Zavedení nové sady tokenizačních parametrů

Tento proces iniciuje *tokenizační autorita*:

1. předem dohodnutým komunikačním kanálem (tel./email) předá provozovateli tokenizačního procesora potřebné údaje pro zavedení nové sady tokenizačních parametrů:
 - Verze tokenu
 - Specifikaci algoritmu pro tokenizaci
 - Parametry pro generování tokenizačního klíče (délka klíče)
 - Platnost tokenizačního klíče (platnost od, platnost do)

2. Provozovatel tokenizačního procesora v rámci založení nové sady tokenizačních parametrů provede vygenerování tokenizačního klíče uvnitř HSM
3. Provozovatel tokenizačního procesora zajistí bezpečnou cestou distribuci vygenerovaného tokenizačního klíče všem dopravcům, kteří jsou zapojeni do systému MOS
4. V případě, že tokenizační autorita stanoví nový, dosud nepoužívaný tokenizační algoritmus, provede provozovatel tokenizačního procesora úpravu systému (nový tokenizační algoritmus bude vyvinut a nasazen v rámci aktualizace systému).
5. Pro nově zavedenou sadu tokenizačních parametrů provede tokenizační procesor pro všechny dosud platné identifikátory *dotokenizaci*, a to v předstihu před začátkem platnosti nového tokenizačního klíče.

Tokenizační procesor předává v rámci dotokenizace nově vypočtený token do backendu MOS. Podobně jako u dávkové tokenizace jsou hodnoty do backendu MOS předávány postupně, po menších objemech, v případě nedostupnosti backendu MOS se data předají po obnovení spojení.

Tokenizační procesor předává do backendu MOS vždy dvojici původní token + nový token tak, aby bylo možné přiřadit nový token k existujícímu identifikátoru v backendu MOS.

Zneplatnění tokenizačního klíče

V případě vyzrazení tokenizačního klíče spouští *tokenizační autorita* proces *zneplatnění tokenizačního klíče*:

1. informuje předem dohodnutým komunikačním kanálem (tel./email) provozovatele tokenizačního procesora o požadavku na zneplatnění
2. stav tokenizačního klíče změní provozovatel tokenizačního procesoru na *invalid* (životní cyklus tokenizačního klíče viz kap. [Tokenizační klíč](#))

Zneplatněný tokenizační klíč se již nebude používat pro tokenizaci, tokenizace nových identifikátorů se bude provádět vždy jen pro *aktivní* tokenizační klíče.

Tokenizace

Tokenizační algoritmus

Na základě dostupných informací a po konzultacích se zadavatelem bude tokenizační řešení umět pracovat s následujícími tokenizačními algoritmy. Vzhledem k odlišnostem (první z popisovaných algoritmů existuje a pravděpodobně nemá v hodnotě tokenu zakódovanou svou

“verzi”) doporučujeme odlišit oba algoritmy délkou výsledného tokenu, tzn. stanovit odlišnou délku tokenu u druhého z nich (viz dále).

Tokenizační algoritmus - verze 1

Tokenizační algoritmus verze 1 je převzat od dopravního podniku, specifikace je uvedena v souboru DP_PDOS_ALG.PDF (viz příloha k tomuto návrhu API).

Upřesnění algoritmu:

- Vstupní data pro BPK budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro Lítačku obsahují první byte typ identifikátoru (shodné s návrhem tokenizačního algoritmu verze 16, viz dále, zde pro lítačku to bude 01), následovat bude UID (délka 7 bytes)
- Vstupní data pro InKartu obsahují CLN (délka 18 znaků) a expiraci ve formátu MMY (4 znaky), vstupní data budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro Mobilní aplikaci obsahují první byte typ identifikátoru (shodné s návrhem tokenizačního algoritmu verze 16, viz dále, zde pro mobilní aplikaci to bude 04) následovat budou DeviceID a AccountID, vstupní data budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro OpusCard jsou identická jako pro typ InKarta

Příklad tokenizace BPK (číslo karty 1234567890123456789, expirace 0116), Lítačky (UID 0A0B0C0D0E0F10), InKarty (CLN 920311200123456781, expirace 0519) a Mobilní aplikace (DeviceID ed229072-46e7-4d8b-a692-cc0bfb852da1 a AccountID accountIdIdentifierExample1) pro tokenizační algoritmus verze 1 pro testovací tokenizační klíč, včetně parametrů vstupujících do encrypt operace a výstupu, hodnoty, které vstupují do výpočtu jako pole bytes, jsou uvedeny v HEX.

Token key:

F0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFFFF0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFFF

encrypt input: 3132333435363738393031323334353637383930313136

encrypt iv: 00000000000000000000000000000000

encrypt alg: AES/CBC/PKCS7PADDING

encrypt output:

5EEBAEE9B7F5B457E4DD7C08410C7F3DB4302AF22967057A3FF08E221DE81537

hash alg: SHA-256

BPK token:

C19C3E064B052951FD12A450D06991C0C91DF119A1F06AC746E1EE0712594407

encrypt input: 010A0B0C0D0E0F10

encrypt iv: 00000000000000000000000000000000

encrypt alg: AES/CBC/PKCS7PADDING

encrypt output: 3376AEBEF5513E23CF40E3F9F9EB3211


```

hash alg:          SHA-256
Litacka token:
33E3A5D37B5B8E5DAC81B8C390F5D2D32F3091E90B7A7B48B941759AB87EF6BE
---
encrypt input:     39323033313132303031323334353637383130353139
encrypt iv:        00000000000000000000000000000000
encrypt alg:       AES/CBC/PKCS5Padding
encrypt output:
3562D62AC8976677267D06CAC8BE261AF94B1E1C2BFB6E36056A54EAD1B82EBA
hash alg:          SHA-256
InKarta token:
3EFF0E178AAEA6FB1C9822F7E2C5A0E2A783AFC0F6BA5A67BB950D5C0077AD1
---
encrypt input:
0465643232393037322D343665372D346438622D613639322D63633062666238353264613161636
36F756E744964656E7469666965724578616D706C6531
encrypt iv:        00000000000000000000000000000000
encrypt alg:       AES/CBC/PKCS7PADDING
encrypt output:
4C59D8ED2044E4AECA4108DE29024E7BF5B95E4E721B526878CAE919FEBE9F4A25E916AC3AF210F
34D552DCE1154A426C7A77AE3B7541324B75D34E418AFF6A7
hash alg:          SHA-256
mobapp token:
CF921C9CB5E98B4D6FA6D4EB894BD38E87DA675E3DB63CD98889949B2E3B569A

```

Výsledná délka tokenu je 32 bytes = 64 hex znaků.

Celkovou délku tokenu lze zkrátit při zvážení možného rizika kolize

(např. 15 bytes = 30 hex znaků - tato délka je uváděna i v následujících příkladech v návrhu API).

Před spuštěním systému tokenizačního řešení v produkčním prostředí bude zaveden do HSM tokenizační klíč nutný pro výpočet tokenu verze 1. Tokenizační klíč bude bezpečným způsobem předán od GPE (správce klíčů pro dopravní podnik) dodavateli Monet+.

V rámci prodejních terminálů a odbavovacích zařízení bude tokenizační klíč i vlastní tokenizační algoritmus uložen a distribuován v rámci dodaného SAM. Tímto způsobem bude zajištěna bezpečnost uloženého klíče a práce s tímto klíčem při výpočtu tokenu uvnitř SAM.

Tokenizační algoritmus - verze 16

Hodnota tokenu bude vypočtena jako

token=*token*_{version} // hash(***encrypt***(*plaintext*,*key*_{*token*}))

tokenversion – obsahuje verzi tokenu – zde 10 hex (16 dec)

znak // označuje zřetězení

hash – hashovací funkce SHA256

encrypt – zašifrování vstupních dat symetrickou blokovou šifrou 3DES

keytoken – tokenizační klíč

plaintext = inputdata // FFpadding na násobek 32

inputdata = type // value1 // value2

type – 1 byte (typ identifikátoru: 0x01-Lítačka, 0x02-BPK, 0x03-InKarta, 0x04-Mobilní aplikace, 0x05-OpusCard), slouží k rozlišení stejných hodnot identifikátorů vydávaných různými dopravci (výsledkem bude rozdílný token)

Plnění hodnot *value1* a *value2* pro jednotlivé typy identifikátorů:

Lítačka:

value1 – UID

Inkarta:

value1 – CLN (18 číslic), **value2** – EXP (ve formátu MMY)

BPK:

value1 – PAN (13-19 číslic), **value2** – EXP (ve formátu MMY)

Mobilní aplikace:

value1 – DeviceID (pravděpodobně 36 znaků, UUID), **value2** – AccountID (string)

OpusCard:

value1 – CLN (18 číslic), **value2** – EXP (ve formátu MMY)

Pozn: formát expirace změněn dne 28.2. z YYMM na MMY tak, aby byl shodný s tokenizačním algoritmem pro DPP.

Upřesnění algoritmu:

- Vstupní data pro BPK (value1 a value2) budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro Lítačku (value1) obsahují UID (délka 7 bytes)
- Vstupní data pro InKartu (value1 a value2) budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro Mobilní aplikaci (value1 a value2) budou do pole bytes vstupující do šifrovací funkce převedena z ASCII
- Vstupní data pro OpusCard (value1 a value2) budou do pole bytes vstupující do šifrovací funkce převedena z ASCII

Příklad tokenizace BPK (číslo karty 1234567890123456789, expirace 0116), Lítačky (UID 0A0B0C0D0E0F10), InKarty (CLN 920311200123456781, expirace 0519) a Mobilní aplikace (DeviceID ed229072-46e7-4d8b-a692-cc0bfb852da1, AccountID accountIdentifierExample1) pro tokenizační algoritmus verze 16 pro testovací tokenizační klíč, včetně parametrů vstupujících do encrypt operace a výstupu, hodnoty, které vstupují do výpočtu jako pole bytes, jsou uvedeny v HEX.

```
Token key:      F0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFFF0F1F2F3F4F5F6F7
---
encrypt input:
023132333435363738393031323334353637383930313136FFFFFFFFFFFFFFFF
encrypt iv:      0000000000000000
encrypt alg:     DESede/CBC/NoPadding
encrypt output:
6E30552AD3D889EB66FF4D8A2C76C7DB9DFEB68587289FC16F110C1376E0A0E3
hash alg:        SHA-256
BPK token:
105C6E76B3C78D05D48E1AB6C08C26C2E24ADEB928BE1E6B3547D19208B9C03BBD
---
```

```
encrypt input:
010A0B0C0D0E0F10FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
encrypt iv:      0000000000000000
encrypt alg:     DESede/CBC/NoPadding
encrypt output:
82FD816355CF3AE2BEC082035583FAE8889390338A662880916901F154899BE9
hash alg:        SHA-256
Litacka token:
109C20F6C6D103544FE54BCF3D84A290CAAEE418542C73AD06578CD48A7AC1CC77
---
```

```
encrypt input:
0339323033313132303031323334353637383130353139FFFFFFFFFFFFFFFF
encrypt iv:      0000000000000000
encrypt alg:     DESede/CBC/NoPadding
encrypt output:
957FC257DE118030131EDC05002C191839EB0E6E795B3D77B7331DF3BC0145ED
hash alg:        SHA-256
InKarta token:
1058C07B1419C48B66DB4C3595183A04CBCB4034F33ABF30651921993F49930910
---
```

```
encrypt input:
0465643232393037322D343665372D346438622D613639322D63633062666238353264613161636
36F756E744964656E7469666965724578616D706C6531FFFF
encrypt iv:      0000000000000000
encrypt alg:     DESede/CBC/NoPadding
encrypt output:
0EB07C89595649474461270AA7938F554FB1A79D14CE6BCC7574F5991CEB986619E136CE87B90CF
9250B7C7D866E9C2AFE4781C1C7DACAFFE8174B65EF1EBDF7
hash alg:        SHA-256
```

mobapp token:

10D86FE6C85940C1769BDF4534D0EC4F9AC0BFF7A656D0CB2D3CB701BF40115211

Hashovací funkce SHA256 a symetrická bloková šifra 3DES mohou být v budoucnu nahrazeny pro budoucí verze tokenizačních algoritmů vhodnějšími variantami.

Výsledná délka tokenu je 33 bytes = 66 hex znaků.

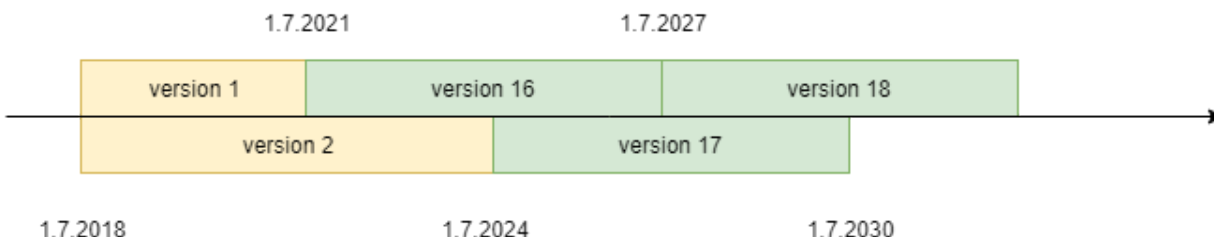
Celkovou délku tokenu lze zkrátit při zvážení možného rizika kolize

(např. 1+15 bytes = 32 hex znaků - tato délka je uváděna i v následujících příkladech v návrhu API).

Navržená metoda tokenizace umožňuje, aby práce s tajným klíčem (zde operace **encrypt**) byla prováděna uvnitř bezpečného hardware terminálu (anebo HSM v rámci tokenizačního procesoru) bez rizika kompromitace tokenizačního klíče. Z pohledu prodejních terminálů a odbavovacích zařízení tento algoritmus nevyžaduje použití SAM pro provedení tokenizace. Současně umožňujeme z hodnoty tokenu identifikovat, o jakou verzi tokenu se jedná.

Platnost tokenizačních klíčů

Pro verzi 1 bude stanovena platnost tokenizačního klíče na 3 roky, pro následující verze bude stanovena platnost na 6 let, přičemž se platnosti budou překrývat, viz následující schéma - **pro verzi 1 a verzi 2 platí tokenizace pomocí původního algoritmu převzatého od dopravního podniku, verze 16 a další označují tokenizaci pomocí nového algoritmu:**



Dvě souběžné překrývající se sady tokenizačních parametrů jsou navrženy ze dvou důvodů:

- Bezpečnostní – okamžitá možnost zneplatnění jedné sady v případě kompromitace, bez omezení provozu systému (tokenizuje se a ověřuje se pomocí druhé sady)
- Provozní – překryv klíčů umožňuje bezvýpadkový provoz u zařízení, které nedisponují možností adhoc reloadu

API pro tokenizaci

Návrh vychází z principů REST API, rozhraní je dostupné přes HTTPS protokol s nakonfigurovaným HTTP Strict Transport Security, data jsou posílána v JSON formátu.

Přístup k API je povolen pouze po provedení autentizace klientským certifikátem, přenášená data jsou kromě transportního protokolu HTTPS zabezpečena na aplikační úrovni pomocí signature (viz dále).

Operace

Jednotlivé operace jsou implementované pomocí následujících HTTP metod:

POST - Vytvoření zdroje / entity

GET - Vyčtení zdroje / entity

PUT - Aktualizace existujícího zdroje / entity

DELETE - Smazání zdroje / entity

Návratové kódy

Rozhraní vrací standardní HTTP response kódy odpovídající výsledku dané operace:

200 OK - Požadavek byl úspěšný, standardní odpověď.

201 Created - Zdroj vytvořen

204 No content - Zdroj smazán

400 Bad Request - Požadavek nemůže být vyřízen. Chyba syntaxe nebo nepovolená operace.

401 Unauthorized - Přístup odepřen

404 Not Found - Zdroj nebyl nalezen

405 Method Not Allowed - Nepovolená metoda požadavku

406 Not Acceptable

429 Too Many Requests - Příliš mnoho požadavků na server, omezení spojení

500 Internal Server Error - Interní chyba serveru

503 Service Unavailable - Služba je dočasně mimo provoz

Chybový response v případě 400 Bad request nebo 500 Internal Server Error (v response předáván JSON obsahující položky code, message, auditId?)

Zabezpečení pomocí signature

Každý dopravce mající přístup k API bude mít přidělen *apiKey* a *apiSecret* sloužící k podepsání požadavku. Parametr *apiSecret* bude dopravci předán bezpečnou cestou (osobní předání nebo předání zabezpečeným kanálem) tak, aby nedošlo ke kompromitaci.

Každý požadavek, který bude dopravce posílat na API, musí obsahovat následující HTTP hlavičky:

STP-APIKEY

Obsahuje hodnotu *apiKey* přidělenou dopravci.

STP-TIMESTAMP

Obsahuje timestamp ve formátu unix time, tj. celé číslo reprezentující počet sekund od 1.1.1970 0:00:00 UTC. Případné požadavky dopravce obsahující shodné parametry budou díky odlišnému timestampu obsahovat odlišné hodnoty podpisu.

Hodnota parametru se při přijetí na serveru nijak nekontroluje oproti serverovému času, je povinností dopravce vyplňovat jeho aktuální čas (= nekonstatní, v čase vzrůstající) hodnotu.

STP-SIGNATURE

Podpis požadavku je generován jako SHA256 HMAC inicializovaný pomocí *apiSecret*, vstupní řetězec bude seskládán jako

`timestamp+method+requestPath+body`

- *timestamp* obsahuje hodnotu předávanou v STP-TIMESTAMP
- *method* obsahuje použitou HTTP method v UPPER CASE formátu
- *requestPath* reprezentuje cestu včetně případných query params (začíná lomítkem, neobsahuje protokol, server a port, např. `/api/v1/registration`)
- *body* obsahuje řetězec předávaný v těle požadavku, pokud není použit, do výpočtu se nekládá (typicky pro `GET` požadavky)
- znak `+` reprezentuje operaci zřetězení příslušných hodnot

Server po přijetí požadavku zkontroluje parametry a ověří hodnotu **STP-SIGNATURE**. V případě nevalidního podpisu vrací HTTP response `401 Unauthorized`.

Příklad seskládání vstupního řetězce a výpočtu signature je uveden u detailního popisu jednotlivých API operací v následujících kapitolách.

Datové typy

V popisu jednotlivých operací API je použit datový typ *DateTime*, jedná se o datum a čas ve formátu „2018-01-25T15:43:17.980“, tzn. lokální datum a čas (bez určení časové zóny) s přesností na ms.

Pro datový typ *Boolean* jsou povolené dvě hodnoty: `true` a `false` (bez uvozovek).

Datový typ *Number* je zadáván bez uvozovek jako celé číslo (např. pro verzi tokenu).

Návrh API pro *registrační požadavek pro tokenizační bránu*

POST `/api/v1/registration`

Operace založí registrační požadavek pro tokenizační bránu.

HTTP metoda: **POST**

Content-type: **application/json**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>eshopRedirectUrl</u>	String	URL pro provedení zpětného přesměrování pomocí GET metody z tokenizační brány na eshop dopravce (dopravce musí předat absolutní adresu včetně protokolu)
<u>mode</u>	String	Mód, určující chování tokenizační brány při zadávání a verifikaci identifikátoru. Povolené hodnoty: <i>register</i> , <i>identify</i> . V případě, že bude nastaven mód <i>register</i> , bude brána akceptovat pouze platné (neexpirované) identifikátory a bude provádět verifikaci identifikátoru. V případě, že bude nastaven mód <i>identify</i> , bude brána umožňovat zadání i expirovaného identifikátoru, v tomto módu se nebude provádět verifikace identifikátoru. Mód <i>identify</i> slouží pro dodatečnou registraci identifikátoru pro dohledání dříve realizovaných plateb.
<u>timeToLiveInSec</u>	Number	Doba platnosti založeného registračního požadavku na tokenizační bráně. Po uplynutí platnosti registrační požadavek expiruje (v případě, že již není v koncovém stavu). Hodnota parametru je v sekundách, minimální povolená hodnota 300 (5 min), maximální 1800 (30 min).

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě úspěšně založeného registračního požadavku se vrací HTTP status

201 Created s parametry

Název	Typ	Popis
<u>regId</u>	String	Identifikátor založeného registračního požadavku
<u>gtwUrl</u>	String	URL pro přechod na tokenizační bránu, obsahuje absolutní adresu včetně protokolu, na tuto adresu přesměruje dopravce zákazníka z eshopu pomocí GET redirectu

V případě, že jsou zadány nevalidní parametry, je vrácena odpověď s HTTP status 400 Bad request s parametry

Název	Typ	Popis
-------	-----	-------

<u>code</u>	Number	Kód chyby, viz číselník Chybový kód
<u>msg</u>	String	Popis chyby

Příklad volání (testovací prostředí)

```
curl -X POST https://testapi.kartajede.cz/api/v1/registration \
-H "Content-Type:application/json" \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867520" \
-H "STP-SIGNATURE:..." \
-d
'{"eshopRedirectUrl":"https://eshop.example.cz/redirect-from-gtw?id=123456","mode":"register"}'
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867520POST/api/v1/registration{"eshopRedirectUrl":"https://eshop.example.cz/redirect-from-gtw?id=123456","mode":"register"}
```

Příklad odpovědi (úspěšně založený registrační požadavek)

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{
  "regId": "1801-f2c9238e-7f5e-4fd3-8a0b-19ad4734f9a5",
  "gtwUrl": "https://test.kartajede.cz/gtw/1801-f2c9238e-7f5e-4fd3-8a0b-19ad4734f9a5/"
}
```

Příklad odpovědi (nevalidní vstupní parametry)

```
HTTP/1.1 400 Bad request
Content-Type: application/json; charset=utf-8

{
  "code": 1001,
  "msg": "Missing required input parameter eshopRedirectUrl"
}
```

GET /api/v1/registration/{regId}

Pomocí této operace zjistí dopravce aktuální stav registrační požadavku.

HTTP metoda: **GET**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>regId</u>	String	identifikátor registračního požadavku získaný při založení registračního požadavku (path parametr)

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

Název	Typ	Popis
<u>regState</u>	String	Stav registračního požadavku, viz životní cyklus registračního požadavku uvedený v kapitole Registrační požadavek
<u>created</u>	DateTime	Datum a čas založení registračního požadavku
<u>mode</u>	String	Mód chování tokenizační brány určující způsob akceptace a verifikace hodnoty identifikátoru, viz popis výše u předchozí operace
identType	String	typ registrovaného identifikátoru, vyplněno v případě, že proběhlo zadání hodnoty identifikátorem.
<u>maskCln</u>	String	Maskované číslo karty ve formátu 6+4 (např. "123456****9876"), vyplněno v případě úspěšně ztokenizovaného identifikátoru typu bankovní karta
<u>cardExp</u>	String	Expirace ve formátu MMY (např. "1121"), vyplněno v případě úspěšně ztokenizovaného identifikátoru typu bankovní karta
tokens	Array	Seznam tokenů pro zadaný identifikátor pro příslušné aktivní tokenizační klíče, vyplněno v případě, že proběhla tokenizace - tzn. stav registračního požadavku je <i>tokenization_finished</i> nebo následný (viz životní cyklus registračního požadavku v kapitole Registrační požadavek), struktura položky tokenu je popsána níže.

Struktura položky v seznamu tokenů (parametr *tokens*)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu
<u>tokenVersion</u>	Number	Verze tokenu (verze odkazuje na to, jakým algoritmem a

		jakým tokenizačním klíčem byl token vytvořen)
<u>tokenState</u>	String	Stav tokenu, viz životní cyklus tokenu v kapitole Token
<u>validFrom</u>	DateTime	Datum a čas začátku platnosti tokenu
<u>validTo</u>	DateTime	Datum a čas konce platnosti tokenu
<u>testOnly</u>	Boolean	Příznak, zda je token vygenerován testovacím tokenizačním klíčem a slouží pouze pro test tokenizace

Příklad volání (testovací prostředí)

```
curl -X GET \
https://testapi.kartajede.cz/api/v1/registration/1801-f2c9238e-7f5e-4fd3-8a0b-19ad4734f9a5 \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867525" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867525GET/api/v1/registration/1801-f2c9238e-7f5e-4fd3-8a0b-19ad4734f9a5
```

Příklad odpovědi

Příklad registrace identifikátoru typu Lítačka, tokenizace podle klíče verze 1 (délka výsledného tokenu je 15 B, 30 hex znaků) a tokenizace podle klíče verze 16 (délka tokenu je 16 B, 32 hex znaků). Vzhledem k neznámému algoritmu verze 1 je hodnota tokenu a jeho délka tokenu uvedena jen jako příklad.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
{
  "regState": "res_recv_backend_mos",
  "created": "2018-01-13T23:11:04.023",
  "mode": "register",
  "identType": "litacka",
  "tokens": [
    {
      "token": "54471ee92c04386811dbc7638741fb",
      "tokenVersion": 1,
      "tokenState": "active",
      "validFrom": "2018-01-01T00:00:00.000",
      "validTo": "2021-12-31T23:59:59.999",
      "testOnly": false
    },
    {
      "token": "10d3956f67781ecb0c93c829d30b9335",
      "tokenVersion": 16,
```

```

    "tokenState": "active",
    "validFrom": "2018-01-01T00:00:00.000",
    "validTo": "2024-12-31T23:59:59.999",
    "testOnly": false
  }
]
}

```

Příklad neúspěšné registrace identifikátoru typu BPK (nepodařilo se ověřit bankovní kartu).

```

HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8

```

```

{
  "regState": "verification_failed",
  "created": "2018-01-13T23:11:04.023",
  "mode": "register",
  "identType": "BPK"
}

```

Návrh API pro *jednotlivou tokenizaci*

Pomocí *jednotlivé tokenizace* bude dopravce tokenizovat pouze non-BPK identifikátory (BPK musí z principu zadat zákazník přes tokenizační bránu nebo přes registrační terminál).

POST /api/v1/identifier

Provede uložení a tokenizaci identifikátoru. Typ a hodnotu non-BPK identifikátor předá dopravce na vstupu.

HTTP metoda: **POST**

Content-type: **application/json**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>identType</u>	String	Typ identifikátoru, povolené jsou všechny non-BPK hodnoty, viz číselník Typ identifikátoru
<u>identValue</u>	String	Vstupní data pro tokenizaci obsahující hodnotu identifikátoru (pro Lítačku bude vyplněna hodnota UID, pro InKartu/OpusCard bude vyplněna hodnota CLN zřetěžená s hodnotou expirace ve formátu MMY , pro Mobilní aplikaci bude vyplněna hodnota DeviceID zřetěžená s hodnotou AccountID)

<u>identRef</u>	String	Sekundární hodnota identifikátoru, neúčastní se tokenizace, předává se do backendu MOS (u Lítačky bude vyplněna hodnota CLN, u InKarty/OpusCard bude vyplněno plné datum platnosti ve formátu „dd.mm.yyyy“ – změnový požadavek, původně pro InKartu nebylo vyplněno), u Mobilní aplikace bude vyplněn parametr Device (bude obsahovat název zařízení)
p1	String	Dodatečný parametry pro registraci identifikátoru, v případě Lítačky bude předáván typ Lítačky podle číselníku Typ Lítačky , v případě Mobilní aplikace bude předána délka AccountID (nutné pro rozparsování identValue a extrahování AccountID pro následné poslání do backendu MOS)
p2	String	Dodatečný parametry pro registraci identifikátoru, v případě Lítačky bude předávána „platnost od“ ve formátu „YYYYMMDD“
p3	String	Dodatečný parametry pro registraci identifikátoru, v případě Lítačky bude předávána „platnost do“ ve formátu „YYYYMMDD“

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě nově založeného identifikátoru se vrací HTTP status 201 Created, v případě existujícího identifikátoru se tokenizace neprovádí, vrací se existující aktivní tokeny pro daný identifikátor s návratovým kódem 200 OK, vždy s parametry

Název	Typ	Popis
<u>identState</u>	String	Stav identifikátoru, viz životní cyklus identifikátoru v kapitole Identifikátor
<u>tokens</u>	Array	Seznam tokenů pro příslušné aktivní tokenizační klíče, struktura tokenu je popsána níže.

Struktura položky v seznamu tokenů (parametr *tokens*)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu
<u>tokenVersion</u>	Number	Verze tokenu (verze odkazuje na to, jakým algoritmem a jakým tokenizačním klíčem byl token vytvořen)
<u>tokenState</u>	String	Stav tokenu, viz životní cyklus tokenu v kapitole Token
<u>validFrom</u>	DateTime	Datum a čas začátku platnosti tokenu
<u>validTo</u>	DateTime	Datum a čas konce platnosti tokenu

<u>testOnly</u>	Boolean	Příznak, zda je token vygenerován testovacím tokenizačním klíčem a slouží pouze pro test tokenizace
-----------------	---------	---

V případě, že jsou zadány nevalidní parametry, je vrácena odpověď s HTTP status 400 `Bad request` s parametry

Název	Typ	Popis
<u>code</u>	Number	Kód chyby, viz číselník Chybový kód
<u>msg</u>	String	Popis chyby

Na rozdíl od registrace identifikátoru přes tokenizační bránu nebo přes registrační terminál se v rámci jednotlivé tokenizace neprovádí verifikace hodnoty identifikátoru oproti backendu dopravce. Po dokončení tokenizace se provádí transfer dat do backendu MOS.

V případě nedostupnosti MOS backendu provede tokenizační procesor předání tokenů později, nezávisle na dokončení operace *jednotlivé tokenizace*.

Příklad volání pro Lítačku (testovací prostředí)

```
curl -X POST https://testapi.kartajede.cz/api/v1/identifier \
-H "Content-Type:application/json" \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867530" \
-H "STP-SIGNATURE:..." \
-d '{
  "identType": "litacka",
  "identValue": "044847F9961B80",
  "identRef": "9203110000584071",
  "p1": "1",
  "p2": "20180401",
  "p3": "20200331"
}'
```

Podpis posílaný v `STP-SIGNATURE` hlavičce je vypočten ze vstupních dat:

```
1516867530POST/api/v1/identifier{"identType":"litacka","identValue":"044847F9961B80",
"identRef":"9203110000584071","p1":"1","p2":"20180401","p3":"20200331"}
```

Příklad volání pro InKartu (testovací prostředí)

```
curl -X POST https://testapi.kartajede.cz/api/v1/identifier \
-H "Content-Type:application/json" \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867530" \
-H "STP-SIGNATURE:..." \
-d '{
  "identType": "inkarta",
  "identValue": "9203112001234567810520",
  "identRef": "N/A"
}'
```

Podpis posílaný v `STP-SIGNATURE` hlavičce je vypočten ze vstupních dat:

```
1516867530POST/api/v1/identifier{"identType":"inkarta","identValue":"9203112001234567
810520","identRef":"N/A"}
```

Příklad volání pro Mobilní aplikaci (testovací prostředí)

```
curl -X POST https://testapi.kartajede.cz/api/v1/identifier \
-H "Content-Type:application/json" \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867530" \
-H "STP-SIGNATURE:..." \
-d
'{"identType":"mobapp","identValue":"ed229072-46e7-4d8b-a692-cc0bfb852da1accountIdentifierExample1","identRef":"iPhone XS","p1":"25"}'
```

Podpis posílaný v STP-SIGNATURE hlavičky je vypočten ze vstupních dat:

```
1516867530POST/api/v1/identifier{"identType":"mobapp","identValue":"ed229072-46e7-4d8b-a692-cc0bfb852da1accountIdentifierExample1","identRef":"iPhone XS","p1":"25"}
```

Příklad odpovědi (úspěšně provedená tokenizace nového identifikátoru)

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
```

```
{
  "identState":"active",
  "tokens": [
    {
      "token": "54471ee92c04386811dbc7638741fb",
      "tokenVersion": 1,
      "tokenState": "active",
      "validFrom": "2018-01-01T00:00:00.000",
      "validTo": "2021-12-31T23:59:59.999",
      "testOnly": false
    },
    {
      "token": "10d3956f67781ecb0c93c829d30b9335",
      "tokenVersion": 16,
      "tokenState": "active",
      "validFrom": "2018-01-01T00:00:00.000",
      "validTo": "2024-12-31T23:59:59.999",
      "testOnly": false
    }
  ]
}
```

Příklad odpovědi (existující identifikátor, tokenizace proběhla dříve)

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
{
  "identState":"active",
  "tokens": [
    {
      "token": "54471ee92c04386811dbc7638741fb",
```

```

    "tokenVersion": 1,
    "tokenState": "active",
    "validFrom": "2018-01-01T00:00:00.000",
    "validTo": "2021-12-31T23:59:59.999",
    "testOnly": false
  },
  {
    "token": "10d3956f67781ecb0c93c829d30b9335",
    "tokenVersion": 16,
    "tokenState": "active",
    "validFrom": "2018-01-01T00:00:00.000",
    "validTo": "2024-12-31T23:59:59.999",
    "testOnly": false
  }
]
}

```

Příklad odpovědi (nevalidní vstupní parametry)

```

HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8

```

```

{
  "code": 1002,
  "msg": "Invalid value of input parametr identType: BPK"
}

```

Návrh API pro *dávkovou tokenizaci*

Pomocí *dávkové tokenizace* bude dopravce tokenizovat pouze non-BPK identifikátory (BPK musí z principu zadat zákazník přes tokenizační bránu nebo přes registrační terminál).

Primárně bude rozhraní sloužit pro dávkovou tokenizaci (např. v rámci iniciální migrace dat), sekundárně tokenizační procesor tímto způsobem přijímá a zpracovává soubor obsahující blacklist od dopravce a dále mapovací soubor od dopravce obsahující dodatečné parametry identifikátorů (viz dále).

Dávková tokenizace je navržena pomocí výměny souborů přes zabezpečené sftp. Každý dopravce má přidělené přístupové údaje a vidí jen svá data (má přidělené „své“ adresáře, do kterých ukládá vstupní soubory a po zpracování tokenizačním procesorem vyčítá výstupní soubory).

Proces výměny souborů bude následující:

- Dopravce provede upload vstupního souboru na sftp do *input* adresáře
- Po dokončení uploadu provede vytvoření *checksum* souboru (má stejný název jako vstupní soubor, obsahuje navíc příponu *.checksum*)

- Tokenizační procesor po detekci *checksum* souboru v *input* adresáři provede načtení a zpracování vstupního souboru
- Tokenizační procesor po úspěšném zpracování vstupního souboru provede přesunutí vstupního a odpovídajícího checksum souboru do *archive* adresáře, v případě chyby při zpracování vstupního souboru provede přesunutí vstupního a checksum souboru do *error* adresáře
- V případě, že je očekáván výstup (platí v případě zpracování vstupního souboru s dávkovou tokenizací, neplatí pro zpracování vstupního souboru s mapováním a s blacklisty), umístí tokenizační procesor výstupní soubor do *output* adresáře, poté provede vytvoření *checksum* souboru (má stejný název jako výstupní soubor, obsahuje navíc příponu *.checksum*)
- Dopravce po detekci *checksum* souboru v *output* adresáři provede načtení a zpracování výstupního souboru
- Dopravce po úspěšném zpracování výstupního souboru provede přesunutí výstupního a odpovídajícího checksum souboru do *archive* adresáře, v případě chyby při zpracování výstupního souboru provede přesunutí výstupního a checksum souboru do *error* adresáře
- Vstupní a výstupní soubory budou šifrovány
- Maximální velikost vstupních souborů je omezena na 100 MB, v případě dávkové tokenizace je povinností dopravce rozdělit data do více menších souborů
- V případě chyby při zpracování vstupního souboru bude celý soubor nezpracovaný (odmítnutí celého souboru, zpracování dat se děje transakčně – buď vše nebo nic). Je na dopravci aby opravil vstupní soubor a provedl následný upload na sftp

Návrh formátu souborů pro dávkovou tokenizaci

Specifikace formátu vstupního souboru pro Lítačku: XML formát, pro každý záznam se bude předávat UID a CLN

```
<BATCH count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM cln="9203110000584071" uid="044847F9961B80" />
  <ITEM cln="9203110004043629" uid="04262399AC2280" />
</ITEMS>
</BATCH>
```

Návrh formátu výstupního souboru pro Lítačku:

BATCH/id se přebírá ze vstupu, BATCH/date odpovídá okamžiku generování výstupního souboru, v záhlaví se přenáší parametry klíčů, u *items* jsou doplněné tokeny:

Pozn: do výstupního souboru doplněn parametr *litackaType* (změnový požadavek zapracovaný od verze 1.5)

```
<BATCH count="2" date="2017-10-13T08:15:52.173" id="3">
<KEYS>
  <KEY version="1" validFrom="2017-01-01T00:00:00.000"
    validTo="2017-12-31T23:59:59.999" testOnly="false" />
  <KEY version="16" validFrom="2018-01-01T00:00:00.000"
    validTo="2018-12-31T23:59:59.999" testOnly="false" />
</KEYS>
<ITEMS>
  <ITEM cln="9203110000584071" litackaType="1" uid="044847F9961B80"
    validFrom="2017-01-10T00:00:00.000" validTo="2017-11-25T23:59:59.999" >
    <TOKEN value="..." version="1" />
    <TOKEN value="..." version="16" />
  </ITEM>
  <ITEM cln="9203110004043629" litackaType="1" uid="04262399AC2280"
    validFrom="2018-02-25T00:00:00.000" validTo="2018-10-15T23:59:59.999" >
    <TOKEN value="..." version="1" />
    <TOKEN value="..." version="16" />
  </ITEM>
</ITEMS>
</BATCH>
```

Specifikace formátu vstupního souboru pro InKartu: XML formát, pro každý záznam se bude předávat CLN a expirace ve formátu **dd.mm.yyyy** (změnový požadavek zapracovaný od verze 1.5, původně předáváno jako MMYYY)

Pozn. od verze 1.6 doplněny atributy *validFrom* a *validTo* do elementu *item*

```
<BATCH count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM cln="920311200123456781" exp="12.02.2020" />
  <ITEM cln="920311208765432101" exp="23.05.2019" />
</ITEMS>
</BATCH>
```

Návrh formátu výstupního souboru pro InKartu:

BATCH/id se přebírá ze vstupu, BATCH/date odpovídá okamžiku generování výstupního souboru, v záhlaví se přenáší parametry klíčů, u *items* jsou doplněné tokeny:

```
<BATCH count="2" date="2017-10-13T08:15:52.173" id="3">
<KEYS>
  <KEY version="1" validFrom="2017-01-01T00:00:00.000"
    validTo="2017-12-31T23:59:59.999" testOnly="false" />
  <KEY version="16" validFrom="2018-01-01T00:00:00.000"
    validTo="2018-12-31T23:59:59.999" testOnly="false" />
</KEYS>
```

```

</KEYS>
<ITEMS>
  <ITEM cln="920311200123456781" exp="12.02.2020" >
    <TOKEN value="..." version="1" />
    <TOKEN value="..." version="16" />
  </ITEM>
  <ITEM cln="920311208765432101" exp="23.05.2019" >
    <TOKEN value="..." version="1" />
    <TOKEN value="..." version="16" />
  </ITEM>
</ITEMS>
</BATCH>

```

Návrh formátu souborů obsahujícího blacklist Lítačky

Tokenizační procesor bude blacklist soubory Lítaček využívat k (offline) verifikaci hodnoty identifikátoru, verifikace u Lítačky tedy nebude prováděna pomocí request/response dotazů do backendu dopravce.

Název souboru bude začínat na „blacklist“ (pro snadnější odlišení od souborů pro dávkovou tokenizaci a od mapovacích souborů)

```

<BLACKLIST count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM cln="9203110000584071" uid="044847F9961B80" />
  <ITEM cln="9203110004043629" uid="04262399AC2280" />
</ITEMS>
</BLACKLIST>

```

Bude předáván vždy kompletní blacklist, nebude se jednat o změnu (diff) od předchozí verze. (předpokládá se denní frekvence aktualizace blacklist souborů).

Návrh formátu souborů obsahujícího blacklist InKarty

Tokenizační procesor bude blacklist soubory InKaret využívat k (offline) verifikaci hodnoty identifikátoru, verifikace u InKaret tedy nebude prováděna pomocí request/response dotazů do backendu dopravce.

Název souboru bude začínat na „blacklist“ (pro snadnější odlišení od souborů pro dávkovou tokenizaci a od mapovacích souborů)

```

<BLACKLIST count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM cln="920311200123456781" />
  <ITEM cln="920311208765432101" />

```

```
</ITEMS>
</BLACKLIST>
```

Bude předáván vždy kompletní blacklist, nebude se jednat o změnu (diff) od předchozí verze. (předpokládá se denní frekvence aktualizace blacklist souborů).

V rámci blacklistu se předává pouze CLN (expirace vynechána, InKarta je plně identifikována pouze přes CLN)

Návrh formátu souboru obsahujícího blacklist tokenů

Tokenizační procesor bude zpracovávat blacklist soubory dopravců a bude zveřejňovat na dávkovém rozhraní blacklist obsahující všechny blacklistované identifikátory. Blacklist bude obsahovat seznam tokenů.

Název souboru bude začínat na „token-blacklist“

```
<TOKENBLACKLIST count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM token="..." version="1" />
  <ITEM token="..." version="16" />
</ITEMS>
</TOKENBLACKLIST>
```

Pozn: od verze 1.3 opraven překlep v uzavíracím tagu </TOKENBLACKLIST> (chybně bylo uvedeno </BLACKLIST>)

Bude předáván vždy kompletní blacklist, nebude se jednat o změnu (diff) od předchozí verze. (předpokládá se denní frekvence aktualizace blacklist souborů obsahujících tokeny). Tento soubor bude primárně zpracováván backendem MOS.

Návrh formátu mapovacího souboru pro Lítačku

Tokenizační procesor bude mapovací soubory Lítačky využívat k překladu CLN -> UID v rámci tokenizace Lítaček pomocí tokenizační brány (zákazník zadává vždy CLN, tokenizuje se UID) a dále k dohledání dodatečných parametrů jako je typ lítačky a platnost za účelem odeslání těchto dat (spolu s vypočtenými tokeny) do backendu MOS.

Název souboru bude začínat na „mapping“ (pro snadnější odlišení od souborů pro dávkovou tokenizaci a od blacklist souborů)

```
<MAPPING count="2" date="2017-10-12T09:31:56.373" id="3">
<ITEMS>
  <ITEM cln="9203110000584071" uid="044847F9961B80" type="1"
    validFrom="2018-01-01T00:00:00.000"
    validTo="2022-12-31T23:59:59.999" />
</ITEMS>
```

```
<ITEM cln="9203110004043629" uid="04262399AC2280" type="2"
  validFrom="2018-02-01T00:00:00.000"
  validTo="2023-01-31T23:59:59.999" />
</ITEMS>
</MAPPING>
```

Kde atribut *type* udává typ lítačky (hodnota se v rámci tokenizačního procesoru pouze přenáší spolu s platností do backendu MOS), povolené hodnoty viz číselník [Typ Lítačky](#)

Bude předáváno vždy kompletní mapování, nebude se jednat o změnu (diff) od předchozí verze.

Mapování pro InKartu a OpusCard

Tokenizační procesor nebude zpracovávat žádný mapovací soubor pro InKartu a OpusCard (není potřeba evidovat ani předávat do backendu MOS žádné další parametry)

Návrh API pro *opětovnou verifikaci existujícího identifikátoru*

Opětovná verifikace existujícího identifikátoru je prováděna bez účasti zákazníka (u lítačky je provedena verifikace oproti blacklistu, u BPK verifikace u issuera)

POST /api/v1/identifier/{token}/verify

Operace pro provedení verifikace pro existující identifikátor.

HTTP metoda: **POST**

Content-type: **application/json**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu, pomocí kterého se hledá navázaný identifikátor, pro který bude prováděna verifikace (path parametr)

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě provedené verifikace se vrací HTTP status

200 OK s parametry

Název	Typ	Popis
-------	-----	-------

token	String	Hodnota tokenu, pomocí kterého se hledá navázaný identifikátor, pro který byla provedena verifikace
verifyResultCode	Number	viz číselník Výsledek verifikace existujícího identifikátoru

V případě, že identifikátor pomocí zadáného tokenu není nalezen, je vrácena odpověď s HTTP status 404 Not Found

Příklad volání (testovací prostředí)

```
curl -X POST
https://testapi.kartajede.cz/api/v1/identifier/54471ee92c04386811dbc7638741fb/verify \
-H "Content-Type:application/json" \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867520" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867520POST/api/v1/identifier/54471ee92c04386811dbc7638741fb/verify
```

Příklad odpovědi (identifikátor je platný)

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
```

```
{
  "token": "54471ee92c04386811dbc7638741fb",
  "verifyResultCode": 0
}
```

Příklad odpovědi (identifikátor není platný, obecné zamítnutí)

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
```

```
{
  "token": "54471ee92c04386811dbc7638741fb",
  "verifyResultCode": 1
}
```

Návrh API pro *správu tokenů a identifikátorů*

Každý identifikátor má k sobě přiřazený jeden nebo více tokenů. Přístup k identifikátoru je možné pouze přes některý z jeho tokenů.

GET /api/v1/token/{token}

Operace pro získání údajů pro daný token.

HTTP metoda: **GET**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě, že je token nalezen, je vrácena odpověď s HTTP status 200 OK s parametry

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu
<u>tokenVersion</u>	Number	Verze tokenu (verze odkazuje na to, jakým algoritmem a jakým tokenizačním klíčem byl token vytvořen)
<u>tokenState</u>	String	Stav tokenu, viz životní cyklus tokenu v kapitole Token
<u>validFrom</u>	DateTime	Datum a čas začátku platnosti tokenu
<u>validTo</u>	DateTime	Datum a čas konce platnosti tokenu
<u>testOnly</u>	Boolean	Příznak, zda je token vygenerován testovacím tokenizačním klíčem a slouží pouze pro test tokenizace

V případě, že token *není* nalezen, je vrácena odpověď s HTTP status 404 Not found

Příklad volání (testovací prostředí)

```
curl -X GET https://testapi.kartajede.cz/api/v1/token/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867540" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867540GET/api/v1/token/54471ee92c04386811dbc7638741fb
```

Příklad odpovědi

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
{
  "token": "54471ee92c04386811dbc7638741fb",
  "tokenVersion": 1,
  "tokenState": "active",
  "validFrom": "2018-01-01T00:00:00.000",
  "validTo": "2021-12-31T23:59:59.999",
  "testOnly": false
}
```

PUT /api/v1/token/{token}

Změní stav příslušného tokenu. Aktuálně pouze pro zablokování a odblokování tokenu.

HTTP metoda: **PUT**

Content-type: **application/json**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu (path parametr)
<u>tokenState</u>	String	Nový požadovaný stav tokenu, povolené hodnoty viz životní cyklus tokenu v kapitole Token (request parametr)

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě, že je token nalezen a je provedena změna stavu, je vrácena odpověď s HTTP status 200 OK s parametry

Název	Typ	Popis
<u>tokenState</u>	String	Aktuální (změněný) stav tokenu
<u>updated</u>	DateTime	Datum a čas změny

V případě, že token *není* nalezen, je vrácena odpověď s HTTP status 404 Not found

V případě, že jsou zadány nevalidní parametry anebo není možné provést změnu stavu (nedovolený přechod), je vrácena odpověď s HTTP status 400 `Bad request` s parametry

Název	Typ	Popis
<code>code</code>	Number	Kód chyby, viz číselník Chybový kód
<code>msg</code>	String	Popis chyby

Příklad volání (testovací prostředí)

```
curl -X PUT https://testapi.kartajede.cz/api/v1/token/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867550" \
-H "STP-SIGNATURE:..." \
-d '{"tokenState":"blocked"}'
```

Podpis posílaný v `STP-SIGNATURE` hlavičce je vypočten ze vstupních dat:

```
1516867550PUT/api/v1/token/54471ee92c04386811dbc7638741fb{"tokenState":"blocked"}
```

Příklad odpovědi (úspěšně provedená operace)

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
```

```
{
  "tokenState": "blocked",
  "updated": "2018-01-20T16:04:20.132"
}
```

Příklad odpovědi (nepovolená hodnota vstupního parametru)

```
HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8
```

```
{
  "code": 1002,
  "msg": "Invalid value of input parameter tokenState: blockked"
}
```

Příklad odpovědi (nelze provést operaci, token není ve správném stavu)

```
HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8
```

```
{
  "code": 1003,
```



```
{
  "msg": "Unable to change tokenState to blocked, transition not allowed"
}
```

DELETE /api/v1/token/{token}

Smaže daný token (provede se fyzické smazání tokenu z databáze tokenizačního procesoru), lze smazat pouze token ve stavu *active*, *invalid* a *expired* viz životní cyklus tokenu v kapitole [Životní cyklus](#)

Vstupní parametry (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu (path parametr)

Příklad volání (testovací prostředí)

```
curl -X DELETE https://testapi.kartajede.cz/api/v1/token/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867560" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867560DELETE/api/v1/token/54471ee92c04386811dbc7638741fb
```

Příklad odpovědi

Úspěšné smazání tokenu:

```
HTTP/1.1 204 No Content
```

Token neexistuje:

```
HTTP/1.1 404 Not found
```

Token nelze smazat (daný token je ve stavu *blocked*):

```
HTTP/1.1 400 Bad request
```

```
Content-Type: application/json; charset=utf-8
```

```
{
  "code": 1003,
  "msg": "Unable to delete token, token is blocked"
}
```

Poznámka: pro entitu token není v API definována žádná operace pro vytvoření, token se zakládá automaticky při první tokenizaci v rámci vytvoření a uložení identifikátoru.

GET /api/v1/identifier/{token}?mode={mode}

Operace pro získání údajů pro identifikátor, který je navázán na příslušný token.

HTTP metoda: **GET**

Požadavek (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu, pomocí kterého se hledá navázaný identifikátor (path parametr)
mode	String	Udává rozsah detailu poskytované v odpovědi, nepovinný (query) parametr, povolené hodnoty: activeTokenValues activeTokensDetail allTokensDetail pokud parametr není uveden, je response poskytován v módu activeTokenValues

Odpověď (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě, že je identifikátor pomocí zadaného tokenu nalezen, je vrácena odpověď s HTTP status 200 OK s parametry

Název	Typ	Popis
<u>identState</u>	String	Stav identifikátoru, viz životní cyklus identifikátoru v kapitole Identifikátor
<u>identType</u>	String	Typ identifikátoru, viz číselník Typ identifikátoru
<u>created</u>	DateTime	Datum a čas zaregistrování identifikátoru
updated	DateTime	Datum a čas poslední změny stavu identifikátoru
tokenValues	Array	Pro mód activeTokenValues vrácen seznam hodnot aktivních tokenů
tokens	Array	Pro mód activeTokensDetail nebo allTokensDetail vrácen seznam tokenů, struktura položky tokenu je uvedena níže.

Struktura položky v seznamu tokenů (parametr *tokens*)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu
<u>tokenVersion</u>	Number	Verze tokenu (verze odkazuje na to, jakým algoritmem a jakým tokenizačním klíčem byl token vytvořen)
<u>tokenState</u>	String	Stav tokenu, viz životní cyklus tokenu v kapitole Token
<u>validFrom</u>	DateTime	Datum a čas začátku platnosti tokenu
<u>validTo</u>	DateTime	Datum a čas konce platnosti tokenu
<u>testOnly</u>	Boolean	Příznak, zda je token vygenerován testovacím tokenizačním klíčem a slouží pouze pro test tokenizace

V případě, že token, přes který se identifikátor hledá neexistuje, je vrácena odpověď s HTTP status `404 Not found`

V případě, že vstupní parametry nejsou validní, je vrácena odpověď s HTTP status `400 Bad request` s parametry

Název	Typ	Popis
<u>code</u>	Number	Kód chyby, viz číselník Chybový kód
<u>msg</u>	String	Popis chyby

Příklad volání (testovací prostředí)

```
curl -X GET https://testapi.kartajede.cz/api/v1/identifier/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867545" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v `STP-SIGNATURE` hlavičce je vypočten ze vstupních dat:

```
1516867545GET/api/v1/identifier/54471ee92c04386811dbc7638741fb
```

Příklad odpovědi (defaultní mód *activeTokenValues*)

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
```

```
{
  "identState": "active",
```

```
{
  "identType": "litacka",
  "created": "2018-01-20T10:15:12.653",
  "tokenValues": [
    "54471ee92c04386811dbc7638741fb", "10d3956f67781ecb0c93c829d30b9335"
  ]
}
```

Příklad odpovědi (požadovaný mód *activeTokensDetail*)

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

```
{
  "identState": "active",
  "identType": "litacka",
  "created": "2018-01-20T10:15:12.653",
  "tokens": [
    {
      "token": "54471ee92c04386811dbc7638741fb",
      "tokenVersion": 1,
      "tokenState": "active",
      "validFrom": "2018-01-01T00:00:00.000",
      "validTo": "2021-12-31T23:59:59.999",
      "testOnly": false
    },
    {
      "token": "10d3956f67781ecb0c93c829d30b9335",
      "tokenVersion": 16,
      "tokenState": "active",
      "validFrom": "2018-01-01T00:00:00.000",
      "validTo": "2024-12-31T23:59:59.999",
      "testOnly": false
    }
  ]
}
```

Příklad odpovědi (požadovaný mód *allTokensDetail*)

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

```
{
  "identState": "active",
  "identType": "litacka",
  "created": "2017-12-20T10:15:12.653",
  "tokens": [
    {
      "token": "25471ee92c04386811dbc7638741ca",
      "tokenVersion": 0,
      "tokenState": "expired",
      "validFrom": "2017-01-01T00:00:00.000",
      "validTo": "2018-12-31T23:59:59.999",
    }
  ]
}
```

```

    "testOnly": true
  },
  {
    "token": "54471ee92c04386811dbc7638741fb",
    "tokenVersion": 1,
    "tokenState": "active",
    "validFrom": "2018-01-01T00:00:00.000",
    "validTo": "2021-12-31T23:59:59.999",
    "testOnly": false
  },
  {
    "token": "10d3956f67781ecb0c93c829d30b9335",
    "tokenVersion": 16,
    "tokenState": "active",
    "validFrom": "2018-01-01T00:00:00.000",
    "validTo": "2024-12-31T23:59:59.999",
    "testOnly": false
  }
]
}

```

Příklad odpovědi (nepovolená hodnota vstupního parametru)

```

HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8

```

```

{
  "code": 1002,
  "msg": "Invalid value of input parameter mode: allTokens"
}

```

PUT /api/v1/identifier/{token}

Změní stav příslušného identifikátoru. Aktuálně slouží pro provedení anonymizace identifikátoru [a pro provedení zneaktivnění \(odregistrace\) identifikátoru](#), viz životní cyklus identifikátoru popsany v kapitole [Identifikátor](#).

HTTP metoda: **PUT**

Content-type: **application/json**

Vstupní parametry (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
-------	-----	-------

<u>token</u>	String	Hodnota tokenu, pomocí kterého se hledá navázaný identifikátor (path parametr)
<u>identState</u>	String	Nový požadovaný stav identifikátoru, povolené hodnoty viz životní cyklus identifikátoru v kapitole Identifikátor (request parametr)

Návratové hodnoty (parametry, které budou vráceny vždy, jsou zvýrazněné)

V případě, že je provedena změna stavu identifikátoru, je vrácena odpověď s HTTP status 200 OK s parametry

Název	Typ	Popis
<u>identState</u>	String	Aktuální stav identifikátoru
<u>updated</u>	DateTime	Datum a čas změny stavu

V případě, že token, přes který se identifikátor hledá, neexistuje, je vrácena odpověď s HTTP status 404 Not found

V případě, že vstupní parametry nejsou validní, je vrácena odpověď s HTTP status 400 Bad request s parametry

Název	Typ	Popis
<u>code</u>	Number	Kód chyby, viz číselník Chybový kód
<u>msg</u>	String	Popis chyby

Příklad volání (testovací prostředí)

```
curl -X PUT https://testapi.kartajede.cz/api/v1/identifier/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867550" \
-H "STP-SIGNATURE:..." \
-d '{"identState":"anonymized"}
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867550PUT/api/v1/identifier/54471ee92c04386811dbc7638741fb{"identState":"anonymized"}
```

Příklad odpovědi (úspěšně provedená operace)

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
```

```
{
  "identState": "anonymized",
  "updated": "2018-01-20T10:20:12.653"
}
```

Příklad odpovědi (nepovolená hodnota vstupního parametru)

```
HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8
```

```
{
  "code": 1000,
  "msg": "Invalid value of input parameter identState: blocked"
}
```

Příklad odpovědi (nelze provést operaci, identifikátor není ve správném stavu)

```
HTTP/1.1 400 Bad request
Content-Type: application/json;charset=utf-8
```

```
{
  "code": 1003,
  "msg": "Unable to change identState to anonymized, transition not allowed"
}
```

DELETE /api/v1/identifier/{token}

Smaže identifikátor, který je navázaný na příslušný token (provede se fyzické smazání identifikátoru z databáze tokenizačního procesoru včetně všech navázaných tokenů pro daný identifikátor).

Vstupní parametry (povinné parametry jsou zvýrazněné)

Název	Typ	Popis
<u>token</u>	String	Hodnota tokenu, podle kterého se hledá navázaný identifikátor (path parametr)

Příklad volání (testovací prostředí)

```
curl -X DELETE
https://testapi.kartajede.cz/api/v1/identifier/54471ee92c04386811dbc7638741fb \
-H "STP-APIKEY:exampleApiKey" \
-H "STP-TIMESTAMP:1516867570" \
-H "STP-SIGNATURE:..."
```

Podpis posílaný v STP-SIGNATURE hlavičce je vypočten ze vstupních dat:

```
1516867570DELETE/api/v1/identifier/54471ee92c04386811dbc7638741fb
```

Příklad odpovědi

Úspěšné smazání identifikátoru:

```
HTTP/1.1 204 No Content
```

Token, přes který se identifikátor hledá, neexistuje:

```
HTTP/1.1 404 Not found
```

Číselníky

Typ identifikátoru

Hodnota	Popis
litačka	Identifikátor typu Lítačka
BPK	Typ <i>bezkontaktní platební karta</i> , nelze použít u <i>jednotlivé</i> nebo <i>dávkové tokenizace</i> (identifikátor typu BPK lze registrovat pouze před registrační terminál nebo přes tokenizační bránu)
inkarta	Identifikátor typu InKarta
mobapp	Identifikátor typu Mobilní aplikace
opuscard	Identifikátor typu OpusCard

Další typy budou dospecifikovány zadavatelem v rámci procesu *Zavedení nového typu identifikátoru*.

Typ Lítačky

Hodnota	Popis
---------	-------

1	Anonymní Lítačka
2	Odpersonalizovaná Lítačka (osobní údaje jsou vytištěné na nosiči, v systémech MOS se neevidují)
3	Personalizovaná Lítačka

Typ BPK

Hodnota	Popis
C	Kreditní platební karta
D	Debetní platební karta
P	Prepaid platební karta

Chybový kód

Kód	Popis
1001	Chybějící povinný vstupní parametr (v msg parametru je uveden detailnější popis chyby, o jaký parametr se jedná)
1002	Nevalidní hodnota vstupního parametru (v msg parametru je uveden detailnější popis chyby, o jaký parametr se jedná)
1003	Nepovolená operace (nelze například provést změnu stavu dané entity)

Výsledek verifikace existujícího identifikátoru

Kód	Popis
0	Verifikace proběhla úspěšně, identifikátor je platný
1	Identifikátor není platný (obecné zamítnutí)
2	Identifikátor je na blacklistu
3	Chyba při ověření identifikátoru (pro BPK identifikátory se např. nepodařilo ověřit kartu u issuera)
4	Platnost identifikátoru vypršela (expirovaná karta)

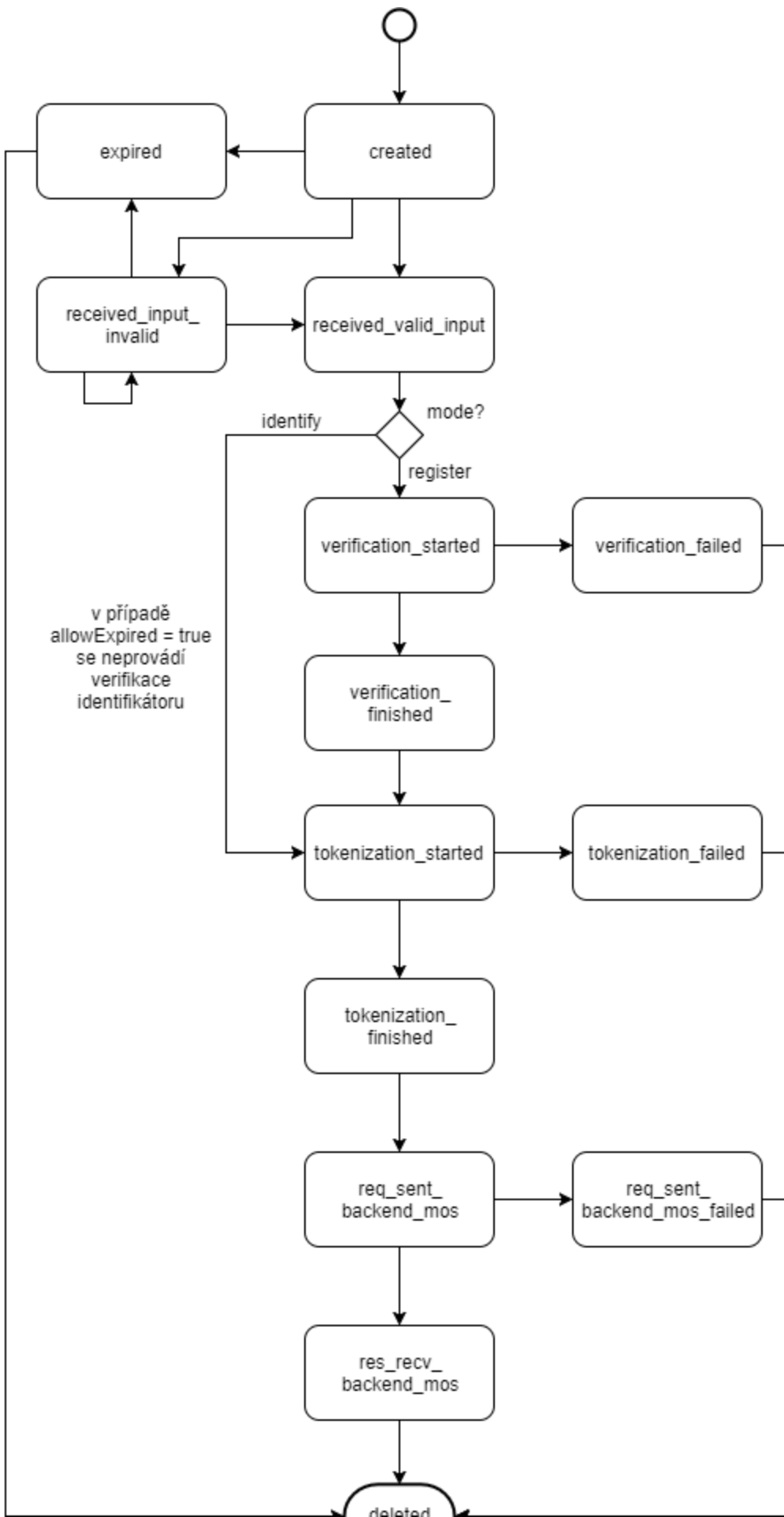
Životní cyklus

Registrační požadavek

Následující schéma popisuje životní cyklus registračního požadavku pro registraci identifikátoru na tokenizační bráně.

Zachycuje jak fázi zadávání hodnoty zákazníkem (s možností opakovaného zadání v případě chyby - překlep / příliš krátké číslo identifikátoru apod), tak následnou verifikaci identifikátoru, jeho tokenizaci a finální odeslání tokenizovaných dat do backendu MOS.

V případě, že zákazník přejde na tokenizační bránu a nedokončí zadání hodnoty, požadavek expiruje. Pokud selže verifikace hodnoty identifikátoru, samotná tokenizace nebo odeslání tokenizovaných dat do backendu MOS, skončí registrační požadavek v odpovídajícím chybovém stavu.



Fyzické smazání registračního požadavku (stav *deleted*) bude provedeno po uplynutí stanovené doby (konfigurační parametr v rámci tokenizačního procesoru) a po splnění příslušných podmínek (= dojde ke smazání jen těch registračních požadavků, ke kterým není navázaný žádný identifikátor, resp. jejichž identifikátor byl již smazán).

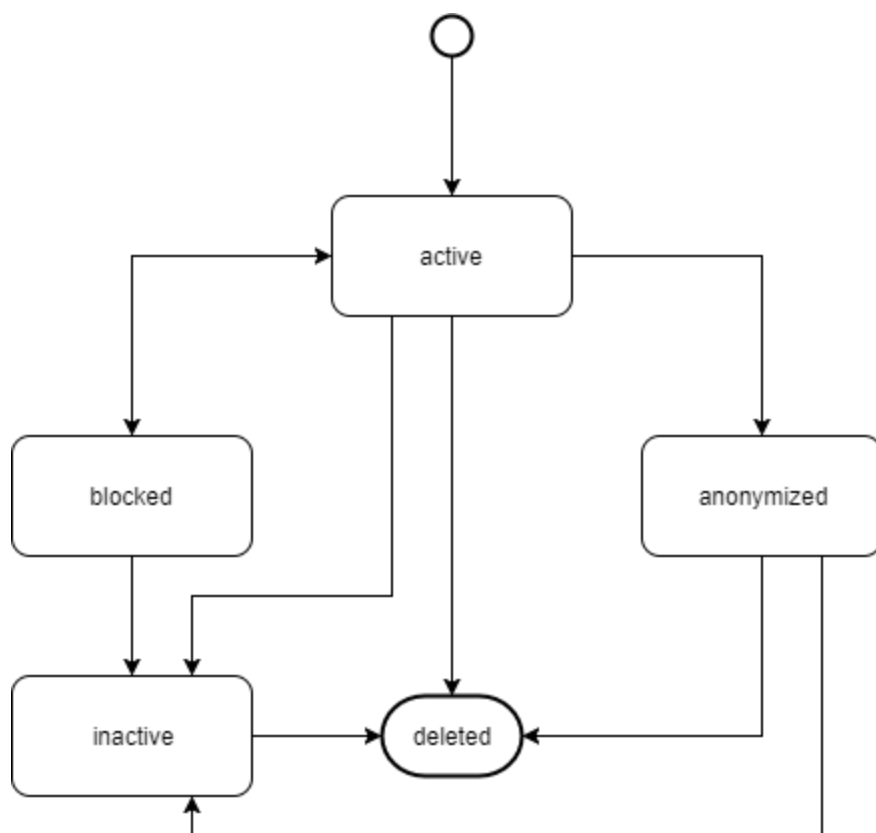
Identifikátor

Každý identifikátor lze pomocí API zablokovat a poté opět odblokovat (např. v situaci, kdy zákazník nahlásí ztrátu identifikátoru), Blokadou se zablokují všechny navázané aktivní tokeny (viz dále). Odblokováním se všechny jeho navázané blokové tokeny změny na aktivní.

V případě zavedení nové sady tokenizačních parametrů se v rámci *dotokenizace* tokenizují vždy všechny aktivní i blokové identifikátory.

Aktivní identifikátor lze anonymizovat (požadavek vyplývající z GDPR týkající se výmazu citlivých dat). Technicky bude provedeno nahrazení hodnoty identifikátoru znaky *****, anonymizovaný identifikátor funguje tak dlouho, dokud budou platit jeho tokeny, které byly aktivní v okamžiku anonymizace (žádné další nové aktivní tokeny pro anonymizovaný identifikátor již nevznikají).

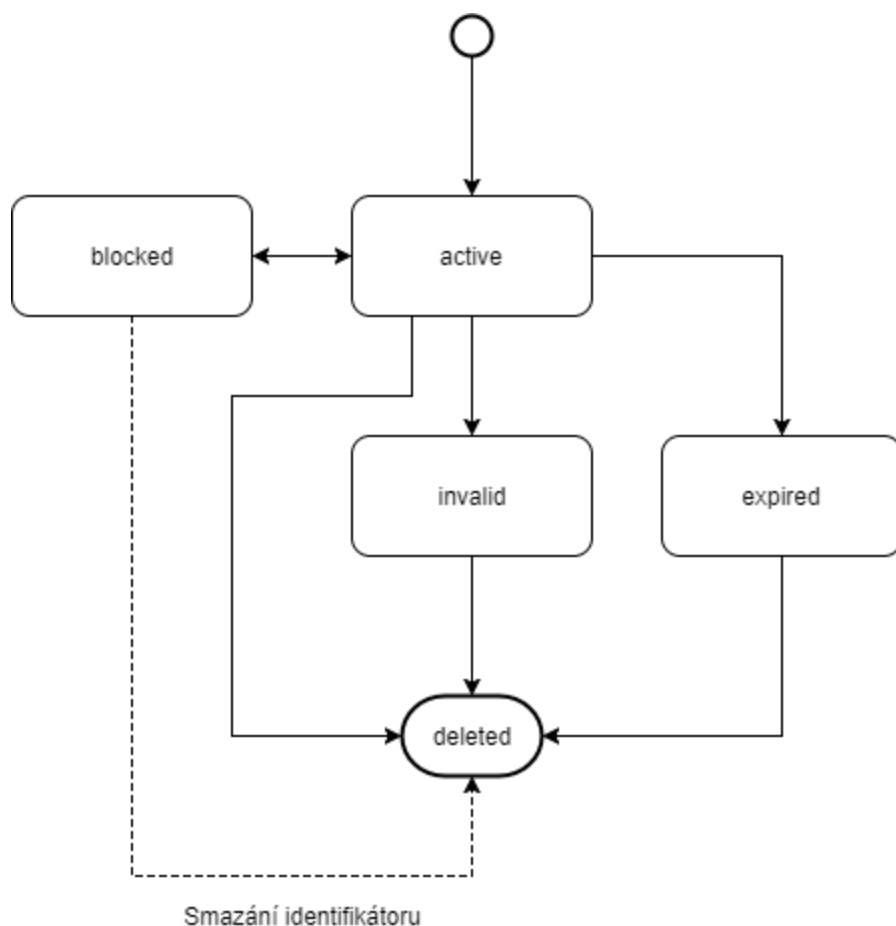
Identifikátor lze zneaktivnit, převést do stavu *inactive* (= provést odregistraci na požadavek zákazníka), zneaktivněním identifikátoru dojde k změně všech navázaných tokenů do stavu *invalid*. Jednou zneaktivněný identifikátor lze pouze fyzicky smazat.



Aktivní, anonymizovaný **a neaktivní** identifikátor lze přes API smazat. Dojde k fyzickému smazání záznamu z databáze, současně s identifikátorem se provede smazání i všech navázaných tokenů (bez ohledu na jejich stav, tzn. budou smazány i blokové tokeny)

Token

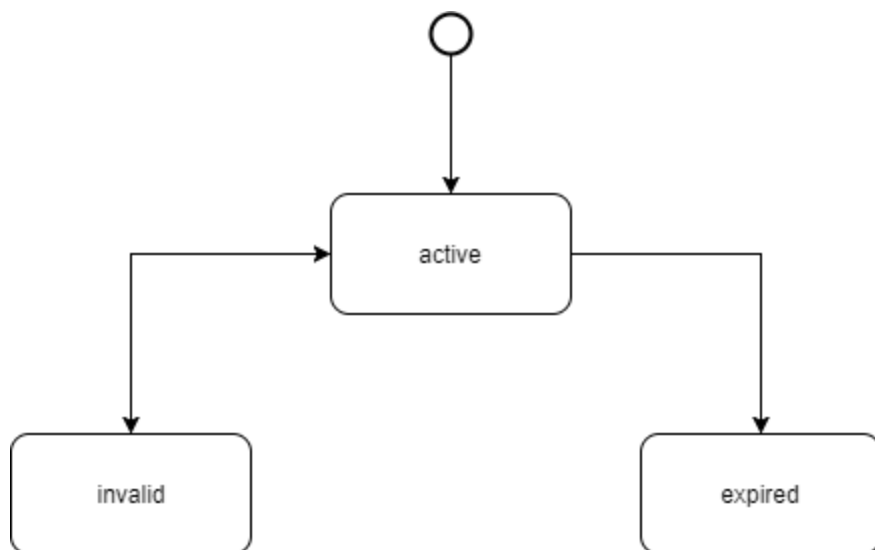
Aktivní token lze přes API zablokovat, v případě zneplatnění tokenizačního klíče budou odpovídající tokeny zneplatněny (stav *invalid*). Po skončení doby platnosti tokenizačního klíče budou odpovídající tokeny expirovány (stav *expired*).



Fyzické smazání záznamu obsahující token lze provést přes API jen pro neblokované tokeny, v případě smazání identifikátoru se mažou všechny navázané tokeny bez ohledu na jejich stav (tzn. i blokové).

Tokenizační klíč

Pro úplnost je uveden i životní cyklus tokenizačního klíče. Nově vygenerovány nebo zavedeny tokenizační klíč je ve stavu *active*, je buď zneplatněn (stav *invalid*) nebo po skončení doby platnosti vyprší (stav *expired*).



Záznamy obsahující tokenizační klíče nejsou fyzicky mazány z databáze.

Tokenizační brána

Tokenizační brána bude navržena s ohledem na responsivní chování a použití na desktopu i mobilních zařízeních. Vlastní zadání identifikátoru na registrační bráně bude provedeno podle následujících pravidel:

Zákazník bude na registračním formuláři mít možnost vybrat, jaký typ identifikátoru bude chtít zadávat, podle tohoto výběru se formulář přizpůsobí a umožní zadat následující typy:

Pro zadávání BPK bude registrační formulář obsahovat pole pro zadání:

- PAN (číslo bankovní karty, délka 13-19 číslic)
- expirace (ve formátu **MMYY**, podle zvoleného módu při založení registračního požadavku kontroluje tokenizační brána expiraci zadané hodnoty)
- ~~CVC-kód~~

Pro zadávání Lítačky bude registrační formulář obsahovat pole pro zadání:

- CLN Lítačky (logické číslo karty, délka 16 číslic, kontrola ~~na Luhn~~, změna od 6/2018, viz následující podkapitola [Validace Lítačky/OpenCard](#))

Pro zadávání InKarty/OpusCard bude registrační formulář obsahovat pole pro zadání:

- CLN (logické číslo InKarty, délka 18 číslic)
- expirace (ve formátu **MMYY**, dd.mm.yyyy - změna 24.5.2018 na základě dohody s OICT tak, aby bylo shodné s datem platnosti vytištěném na fyzickém nosiči, podle zvoleného

módu při založení registračního požadavku kontroluje tokenizační brána expiraci zadané hodnoty)

Po výběru Mobilní aplikace ...

1

Typ identifikátoru

2

Údaje

Pro úspěšné přidání identifikátoru nejdříve vyberte jeho typ.
Následně budete přesměrováni k zadání čísla identifikátoru.

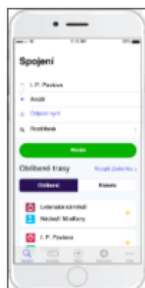
 Mobilní aplikace PID Lítačka	 Lítačka/OpenCard	 Platební karta Visa, MasterCard	 InKarta
--	---	---	--

Zpět

...bude na tokenizační bráně zobrazen návod na použití mobilní aplikace (nebude se zadávat/pokračovat v tokenizaci na tokenizační bráně).

1 Typ identifikátoru ————— 2 Údaje

Mobilní aplikace PIDLítačka jako identifikátor



- 1) Stáhněte si mobilní aplikaci PID Lítačka
- 2) Přihlaste se svým účtem z e-shopu
- 3) V sekci Můj účet-Moje identifikátory v mobilní aplikaci klikněte na "Přidat tento telefon jako identifikátor"

Nyní se z této stránky vraťte zpět a v sekci "Moje identifikátory" na eshopu uvidíte svoje mobilní zařízení jako nově přidáný identifikátor

Aplikaci lze stáhnout zde



ZPĚT NA
IDENTIFIKÁTORY

Po odeslání formuláře bude provedena validace zadaných hodnot, následná verifikace, tokenizace a odeslání tokenizovaných dat do backendu MOS. Z pohledu zákazníka bude zobrazena informace o průběhu zpracování. Po dokončení zpracování bude zákazníkovi zobrazena informace o výsledku zpracování a zákazník bude přesměrován zpět na eshop dopravce.

Pageflow a návrh obrazovek (grafické prvky, popis polí, textace) bude upřesněn po dodání podkladů od zadavatele.

Tokenizační brána bude rozšířena o možnost předání locale (jazyk) při přechodu z eshopu na tokenizační bránu a dále při přechodu zpět z tokenizační brány na eshop (bude řešeno pomocí query parametru *lang*)

Validace Lítačky/OpenCard/OpusCard

Původní validace Lítačky byla implementována pomocí LUHN algoritmu, pojmem „Lítačka“ se souhrnně označují jak původní karty OpenCard, tak nově vydávané karty Lítačka. Algoritmus LUHN byl použit univerzálně pro oba typy.

V rámci změnového požadavku bude tato validace na tokenizační bráně upravena a bude ověřována pomocí následujícího postupu:

- pokud je na sedmé pozici něco jiného než nula, jedná se o Lítačku a validujeme číslo karty podle dodaného algoritmu pro Lítačku (viz dále)
- v případě, že je na sedmé pozici nula, validujeme nejprve na LUHN (pokud vyjde, jedná se o validní číslo OpenCard karty), pokud předchází validace na LUHN neprojde, validujeme pomocí algoritmu pro Lítačku, pokud vyjde, jedná se o validní číslo Lítačky
- v případě, že obě validace v předchozím kroku neprojdou, jedná se o nevalidní číslo karty, tokenizační brána "nepustí uživatele" dále ...

výše uvedené by mělo být pro tokenizační procesor a tokenizační bránu dostačující - pro samotnou tokenizaci potřebujeme jen číslo karty, nepotřebujeme znát, zda se jedná o OpenCard nebo Lítačku (což z druhého kroku 100% nezjistíme - teoreticky může existovat číslo karty, kde by byla na sedmé pozici nula, seděl by LUHN a současně by seděl algoritmus pro Lítačku - pokud ano, nebyli bychom schopni určit, zda se jedná o OpenCard nebo Lítačku)

Nebude použita validace online oproti card managementu.

Pro ověření čísla Lítačky bude použit algoritmus:

$X = \text{SUM}(\text{sudé číslice} \cdot 2) + \text{SUM}(\text{liché číslice})$

$Y = \text{SUM}(\text{číslíky } X)$

Pokud platí $Y \bmod 10 == 0$ je číslo Lítačky validní

Registrační terminál

Registrační terminál bude umístěn na kontaktním místě MOS, nebude propojen s Pokladnou (Obslužnou aplikací), terminál bude komunikovat přes terminal API s tokenizačním procesorem. Aktivace čtečky terminálu bude pomocí hotkey na klávesnici terminálu. Terminál bude obsahovat SAM obsahující potřebné klíče pro verifikaci karet typu Lítačka.

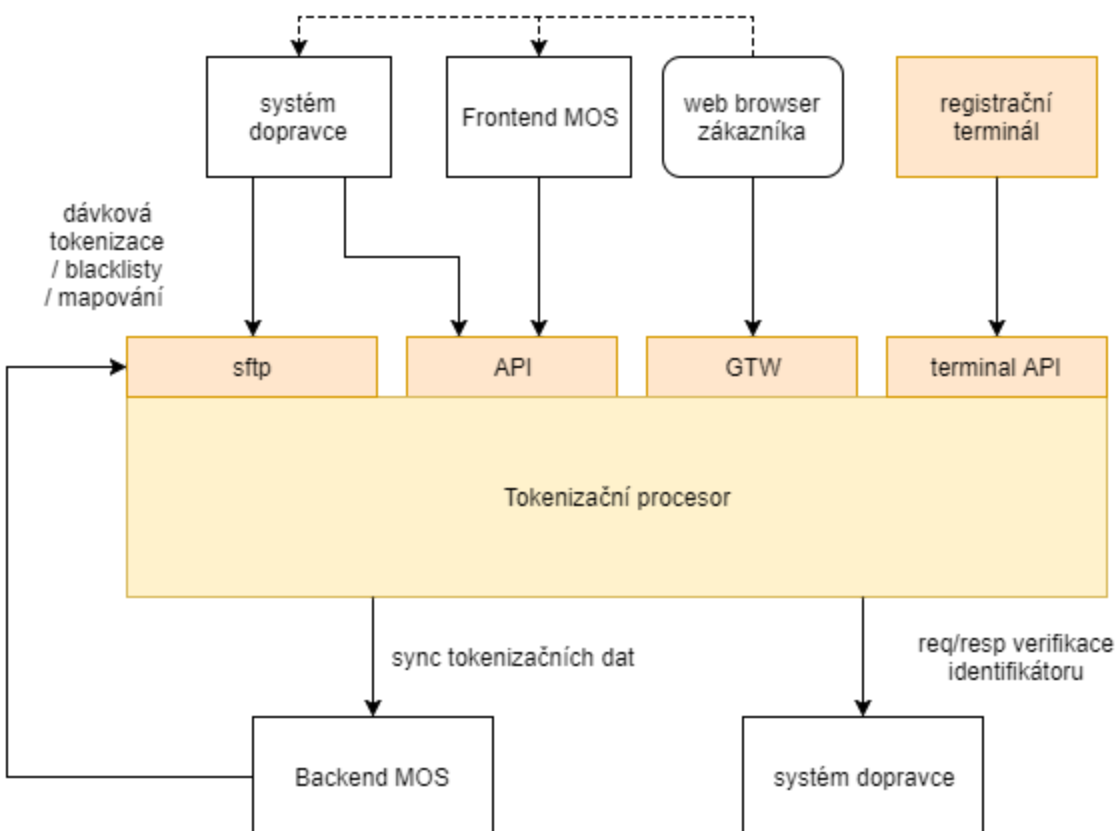
Registrační terminál bude běžet v režimu, kdy bude pouze vyčítat a provádět verifikaci identifikátorů (v případě BPK bude provádět Offline data authentication, v případě Lítačky bude provádět ověření vůči SAM), vlastní tokenizace bude prováděna až na úrovni tokenizačního procesoru, tokenizační procesor je odpovědný za odeslání ztokenizovaných dat dále do backendu MOS (z tohoto důvodu je efektivnější tokenizaci provádět až na serveru).

Architektura systému

Logická architektura

Následující schéma zachycuje blokové členění navrhovaného systému, tokenizační procesor obsahuje moduly realizující čtyři vstupní kanály pro provedení tokenizace:

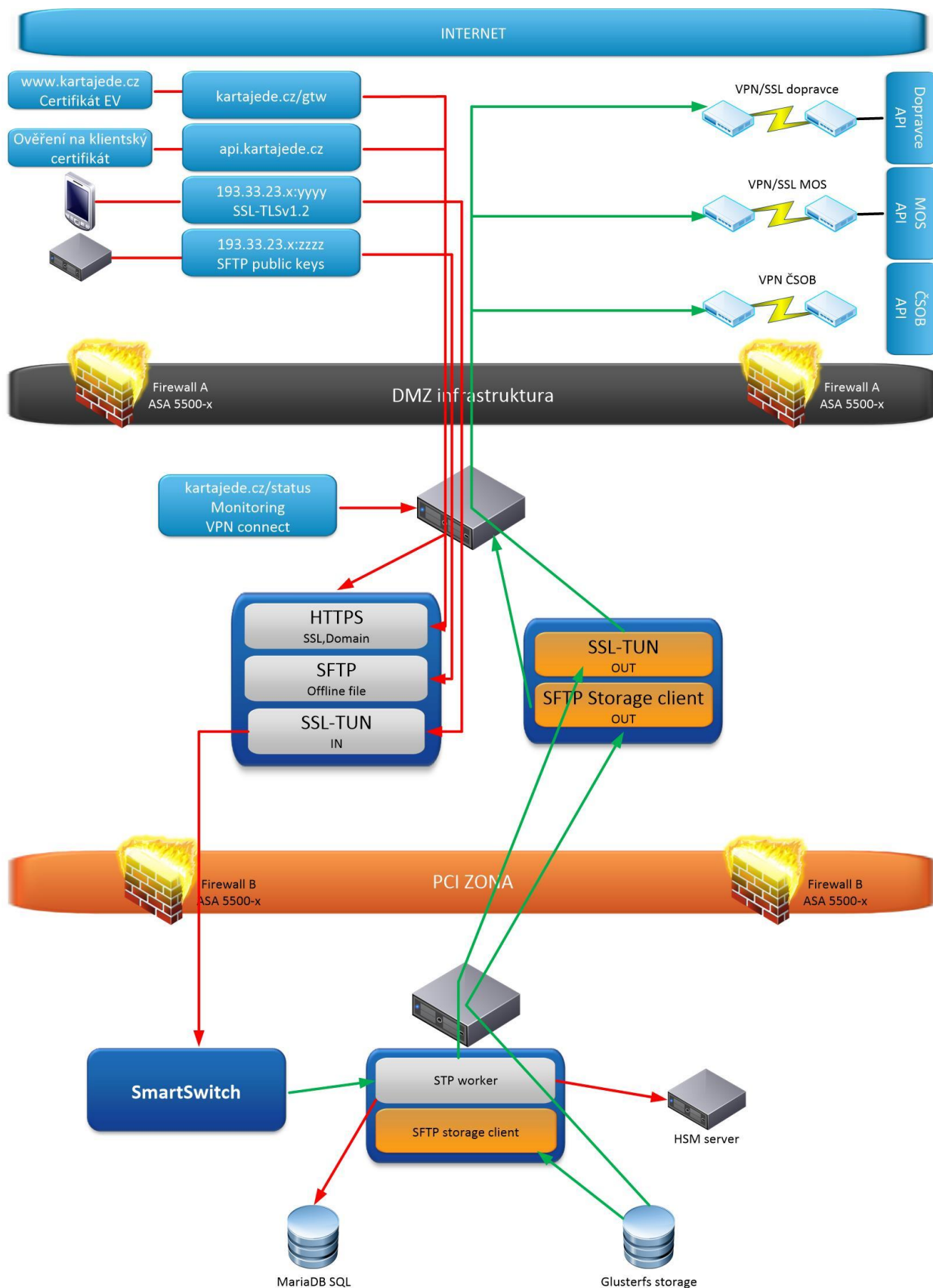
1. sftp pro dávkové rozhraní
2. API pro provedení
 - a. jednotlivé tokenizace
 - b. založení a zjištění stavu registračního požadavku
 - c. správu identifikátorů a tokenů
3. webové rozhraní pro tokenizační bránu (GTW)
4. terminal API pro připojení registračního terminálu pro provedení registrace identifikátoru přes registrační terminál



Tokenizační procesor je napojen na rozhraní backendu MOS (synchronizace tokenizačních dat) a na rozhraní dopravců (pro verifikaci identifikátorů).

Fyzická architektura

Navrhovaný systém tokenizačního řešení bude provozován v prostředí PCI DSS, systém bude provozován v režimu 24x7 (cluster řešení s podporou failover & loadbalancing). Následující schéma obsahuje návrh síťové infrastruktury.



Napojení na okolní systémy

Transfer dat do backendu MOS

Navrhujeme, aby rozhraní pro přenos dat do backendu MOS bylo podobně jako API tokenizačního procesoru zpřístupněno jako REST API, JSON formát. Data, která bude tokenizační procesor odesílat do backendu MOS jsou následující:

Aktuálně bude tokenizační procesor uchovávat hodnoty **čtyř** typů identifikátorů, pro každý z nich bude do backendu MOS předávat jinou množinu parametrů:

BPK identifikátor

- Jeden nebo více tokenů (vždy verze a hodnota tokenu, počet předávaných tokenů bude odpovídat počtu aktivních tokenizačních klíčů, předpokládá se, že souběžně budou dva aktivní tokenizační klíče)
- Maskované číslo karty (6+4)
- Expirace karty (**MMYY**)
- Země vydavatele (dvoupísmenný ISO kód, např. CZ, SK, DE, ...)
- Typ BPK, viz číselník [Typ BPK](#)
- Form factor

Lítačka

- Jeden nebo více tokenů (vždy verze a hodnota tokenu, počet předávaných tokenů bude odpovídat počtu aktivních tokenizačních klíčů, předpokládá se souběžně 2 aktivní tokenizační klíče)
- CLN
- UID
- Typ Lítačky, viz číselník [Typ Lítačky](#)
- Platnost od
- Platnost do

InKarta

- Jeden nebo více tokenů (vždy verze a hodnota tokenu, počet předávaných tokenů bude odpovídat počtu aktivních tokenizačních klíčů, předpokládá se souběžně 2 aktivní tokenizační klíče)
- CLN
- Expirace karty (dd.mm.yyyy) - pozn. předávání platnosti Inkarty změněno od verze 1.5 na formát dd.mm.yyyy (změnový požadavek)

Mobilní aplikace

- Jeden nebo více tokenů (vždy verze a hodnota tokenu, počet předávaných tokenů bude odpovídat počtu aktivních tokenizačních klíčů, předpokládá se souběžně 2 aktivní tokenizační klíče)
- Device (název zařízení posílán v položce maskedPan)
- AccountID (posláno v položce cln)

InKarta

- Jeden nebo více tokenů (vždy verze a hodnota tokenu, počet předávaných tokenů bude odpovídat počtu aktivních tokenizačních klíčů, předpokládá se souběžně 2 aktivní tokenizační klíče)
- CLN
- Expirace karty (dd.mm.yyyy)

Vzhledem ke způsobu zpracování dat na straně tokenizačního procesoru budou platit následující pravidla pro předávání dat do backendu MOS:

Dávková tokenizace

Předávají se jen non-BPK identifikátory a to pouze typ Lítačka, **InKarta**

Jednotlivá tokenizace

Předávají se jen non-BPK identifikátory, tzn. Jen typ Lítačka, **InKarta**, **Mobilní aplikace**, **OpusCard**

Proces dotokenizace

Bude předáván tzv. starý (předchozí verze) a nový (nově tokenizovaný) token - potřebné pro určení, k jakým identifikátorům se nové tokeny vážou

Registrace identifikátoru přes tokenizační bránu

Pro BPK, Lítačku, **InKartu** a **OpusCard** budou předávány výše definované parametry, navíc bude předáván parametr regId (pro spárování registračního požadavku)

Registrace identifikátoru přes registrační terminál

Pro BPK, Lítačku, **InKartu** a **OpusCard** budou předávány výše definované parametry.

~~Přesná specifikace rozhraní zohledňující výše uvedená pravidla je uvedena v souboru TokenTransferWSv1.wsdl (viz příloha k tomuto návrhu API).~~

Specifikace od verze 1.3 změněna ze SOAP WS na REST/JSON rozhraní:
Budou dostupné dvě operace:

identRegistration - POST, v request body bude JSON s následujícím obsahem:

ukázkový request pro Lítačku:

```
{
  "identType": "litacka",
  "regId": "1803-8abe9c3d-d58f-4c78-8336-7429ac782df6",
  "litacka": {
    "cln": "920...",
    "uid": "AB...",
    "validFrom": "",
    "validTo": "",
    "litackaType": 1
  },
  "tokens": [
    {
      "tokenValue": "54471ee92c04386811dbc7638741fb",
      "tokenVersion": 1
    },
    {
      "tokenValue": "10d3956f67781ecb0c93c829d30b9335",
      "tokenVersion": 16
    }
  ]
}
```

ukázkový request pro BPK:

```
{
  "identType": "BPK",
  "regId": "1803-8abe9c3d-d58f-4c78-8336-7429ac782df6",
  "bpk": {
    "maskedPan": "412501****0208",
    "expiry": "1119",
    "issuerCountryCode": "CZ",
    "formFactor": "...",
    "bpkType": "C"
  },
  "tokens": [
    {
      "tokenValue": "54471ee92c04386811dbc7638741aa",
      "tokenVersion": 1
    },
    {
      "tokenValue": "10d3956f67781ecb0c93c829d30b9399",
      "tokenVersion": 16
    }
  ]
}
```

ukázkový request pro InKartu:

Pozn. předávání platnosti Inkarty změněno od verze 1.5 na formát dd.mm.yyyy (změnový požadavek)

```
{
  "identType": "inkarta",
  "regId": "1803-8abe9c3d-d58f-4c78-8336-7429ac782df6",
  "inkarta": {
    "maskedCln": "920320****0208",
```



```

        "expiry": "10.05.2020",
    },
    "tokens": [
        {
            "tokenValue": "54471ee92c04386811dbc7638741aa",
            "tokenVersion": 1
        },
        {
            "tokenValue": "10d3956f67781ecb0c93c829d30b9399",
            "tokenVersion": 16
        }
    ]
}

```

ukázkový request pro Mobilní aplikaci:

Parametr maskedPan obsahuje informaci o Device (název zařízení), parametr cln obsahuje hodnotu AccountID.

```

{
  "identType": "mobapp",
  "regId": "1803-8abe9c3d-d58f-4c78-8336-7429ac782df6",
  "mobapp": {
    "maskedPan": "iPhone XS",
    "cln": "accountIdentifierExample1"
  },
  "tokens": [
    {
      "tokenValue": "54471ee92c04386811dbc7638741aa",
      "tokenVersion": 1
    },
    {
      "tokenValue": "10d3956f67781ecb0c93c829d30b9399",
      "tokenVersion": 16
    }
  ]
}

```

tokenSync - slouží pro předání údajů v případě dotokenizace identifikátoru - POST, v request body bude JSON s následujícím obsahem:

```

{
  "oldTokenValue": "...",
  "oldTokenVersion": 1,
  "newTokenValue": "...",
  "newTokenVersion": 2
}

```

Verifikace identifikátoru pro jednotlivé dopravce

Specifikace rozhraní dopravce bude upřesněno v rámci zavedení nového typu identifikátoru. Pro stávající Lítačku není potřeba (soubor s blacklisty a s mapováním je předáván přes dávkové rozhraní a verifikace je tak prováděna na straně tokenizačního procesoru).

Pro InKartu není potřeba (soubor s blacklisty a s mapováním je předáván přes dávkové rozhraní a verifikace je tak prováděna na straně tokenizačního procesoru).